Lecture 5
PID control & simulation engines

Robot Control · University of Warsaw

Feedback control with three terms, transfer functions, differential equations and how a simulator integrates them, and where the same ideas reappear in machine learning.

Motivation and Overview

What does it take to fly a drone?

1 · differential equations

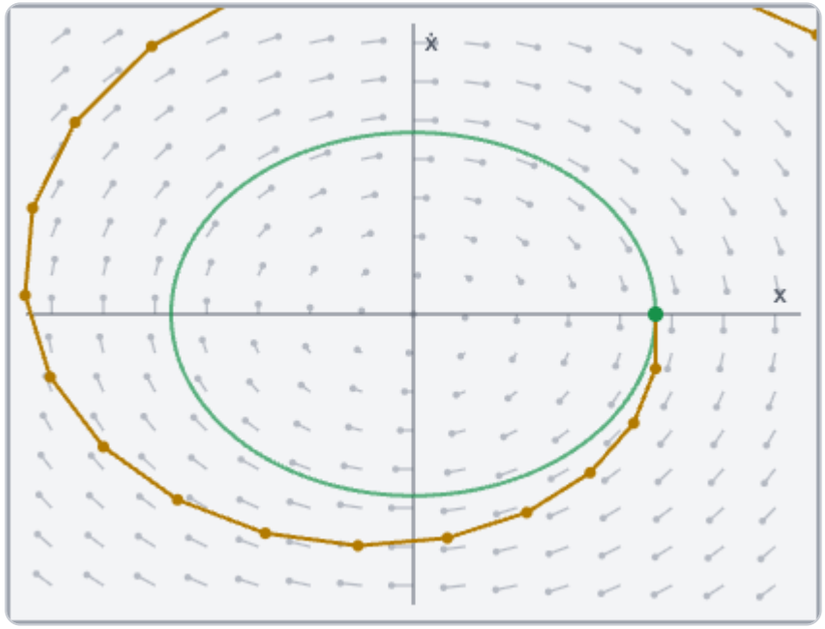
the state of a system, constraints, phase space

2 · PID control

feedback, the three terms on a drone, several controllers at once, transfer functions

3 · simulation engines

numerical integration and its time step, simulating deformation, the same ideas in flow matching



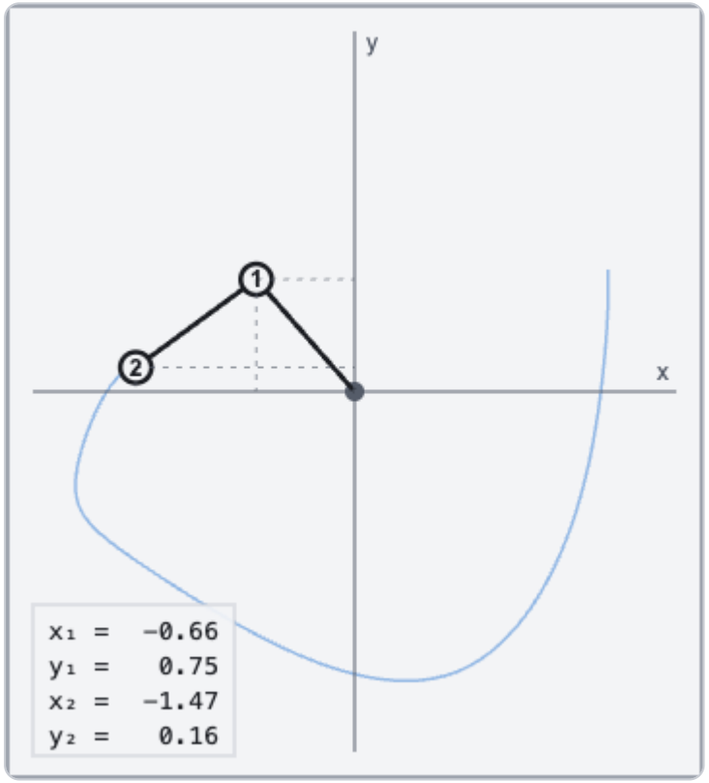
Part 1

Differential equations

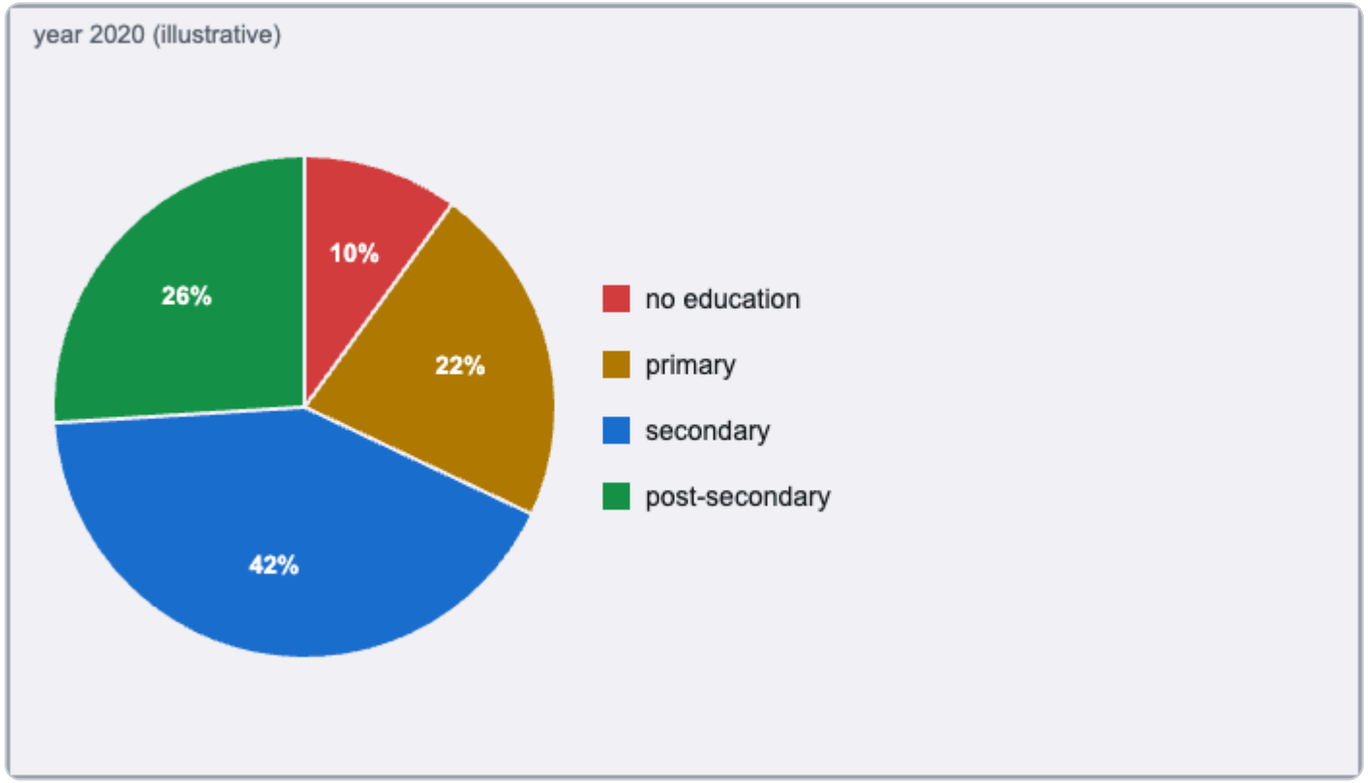
The state of a system

A set $\vec{q}$ of parameters (coordinates) whose values determine the state of the system

A double pendulum



The population of the world by level of education

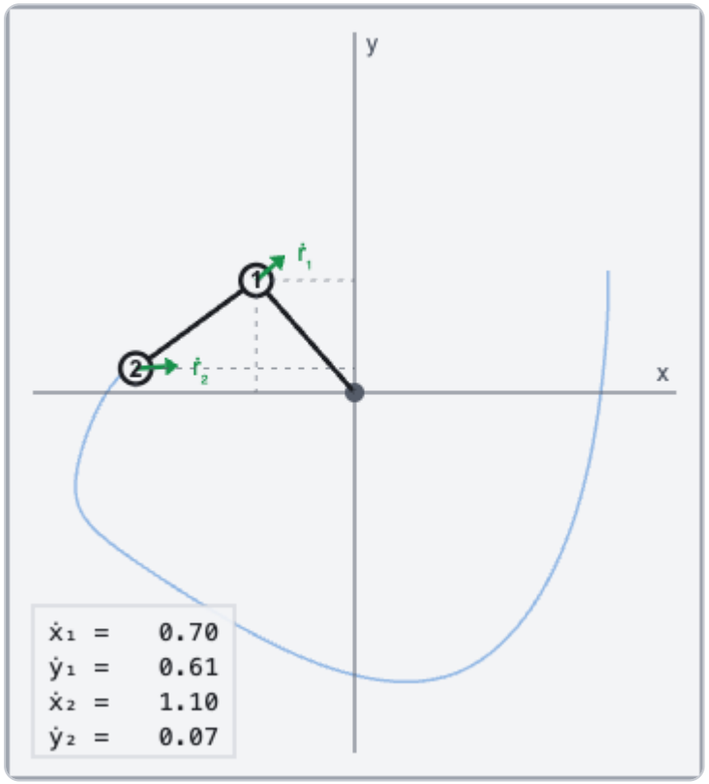


Question How many dimensions does $\vec{q}$ have?

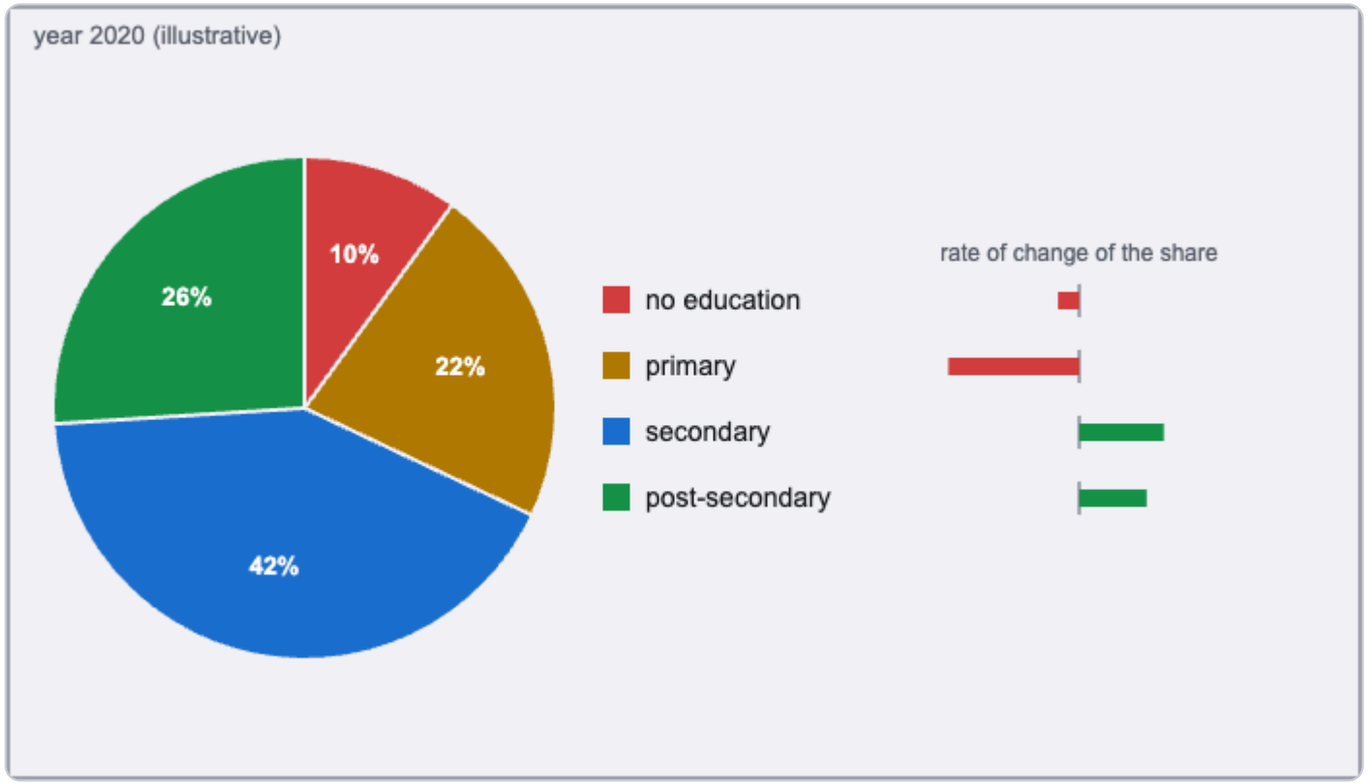
Derivatives describe change

We often refer to derivatives with names from physics: **velocity** $\dot{\vec{q}}$ or **acceleration** $\ddot{\vec{q}}$

A double pendulum



The population of the world by level of education

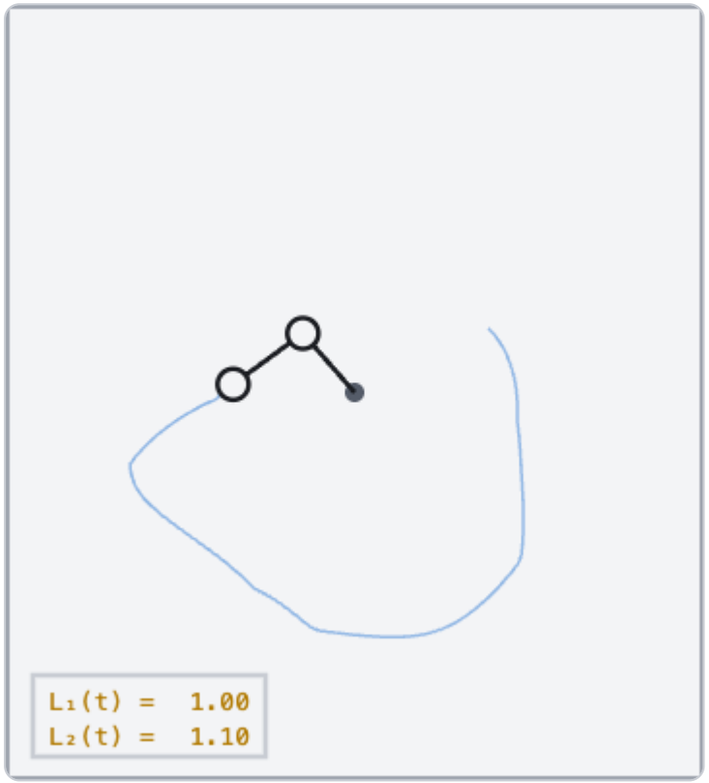


Question And $\dot{\vec{q}}$, how many dimensions?

Constraints

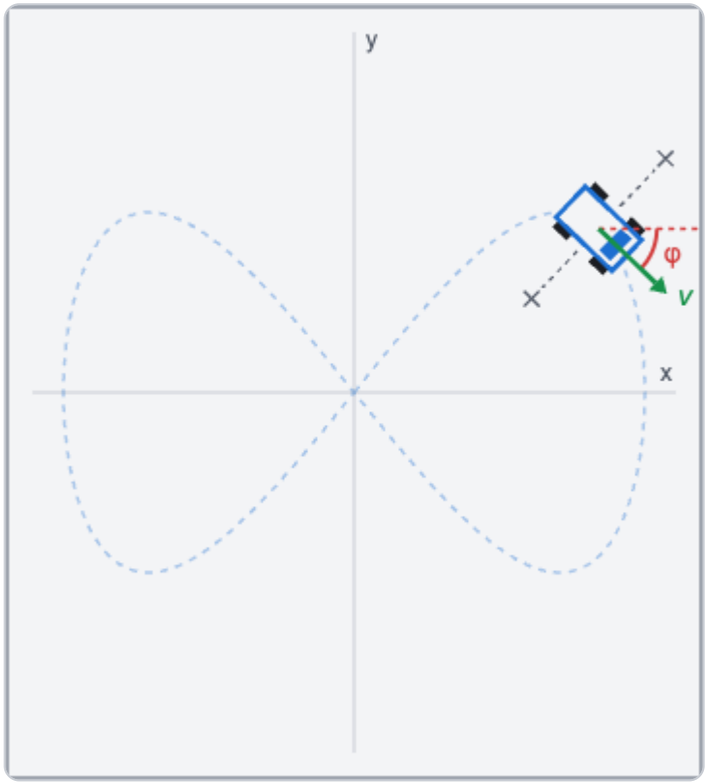
Usually the system has to satisfy **constraints**: the state is somehow restricted $g(\vec{q}, \dot{\vec{q}}, t) = 0$

Holonomic: no derivatives



rod lengths, which may depend on time (e.g., stretching)

Nonholonomic: derivatives



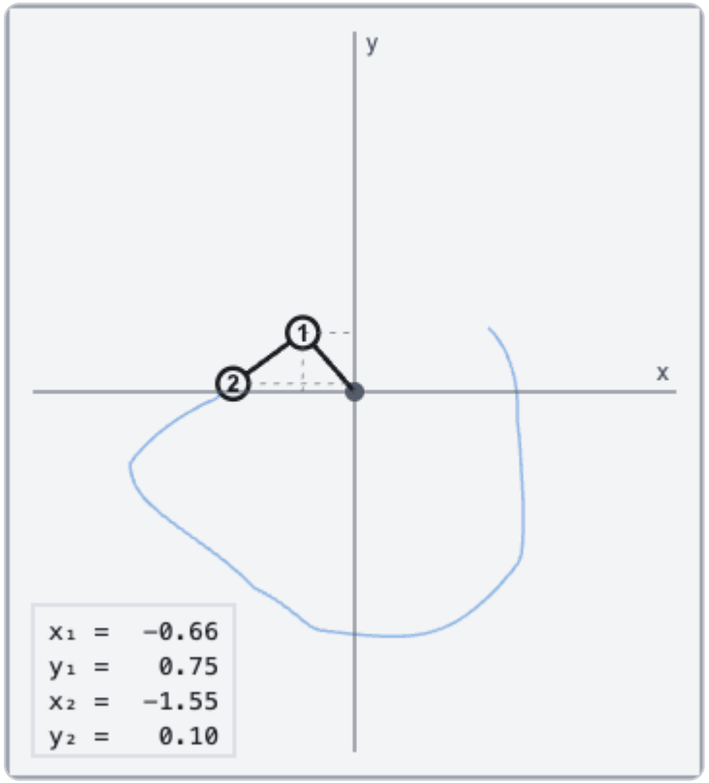
the car cannot move sideways: $\dot{x} \sin \varphi = \dot{y} \cos \varphi$

Generalised coordinates

Degrees of freedom (DoF) = #parameters – #holonomic constraints

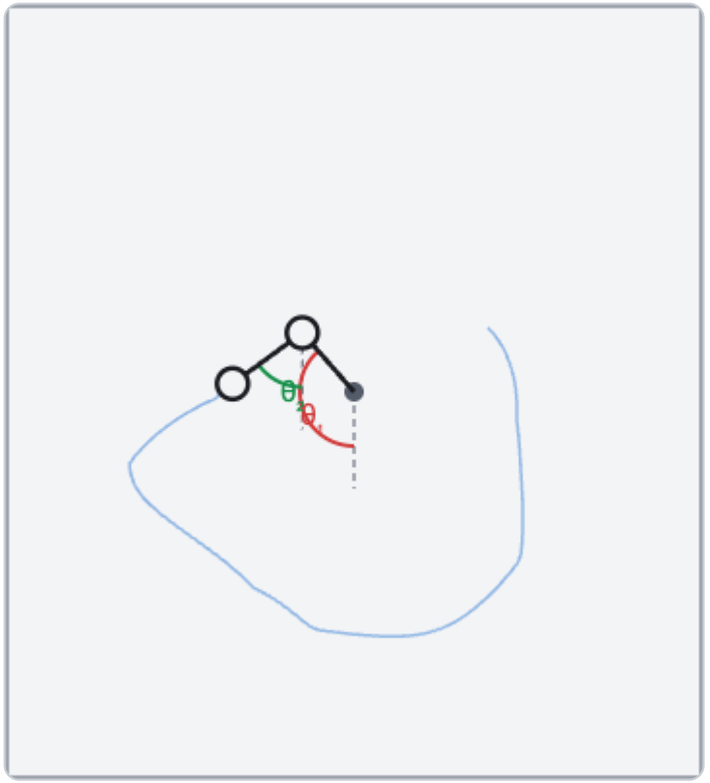
- we can choose **any** set of parameters to describe the system
- often the most convenient one has #parameters = #DoF

A double pendulum



4 coordinates, 2 holonomic constraints

switch the representation



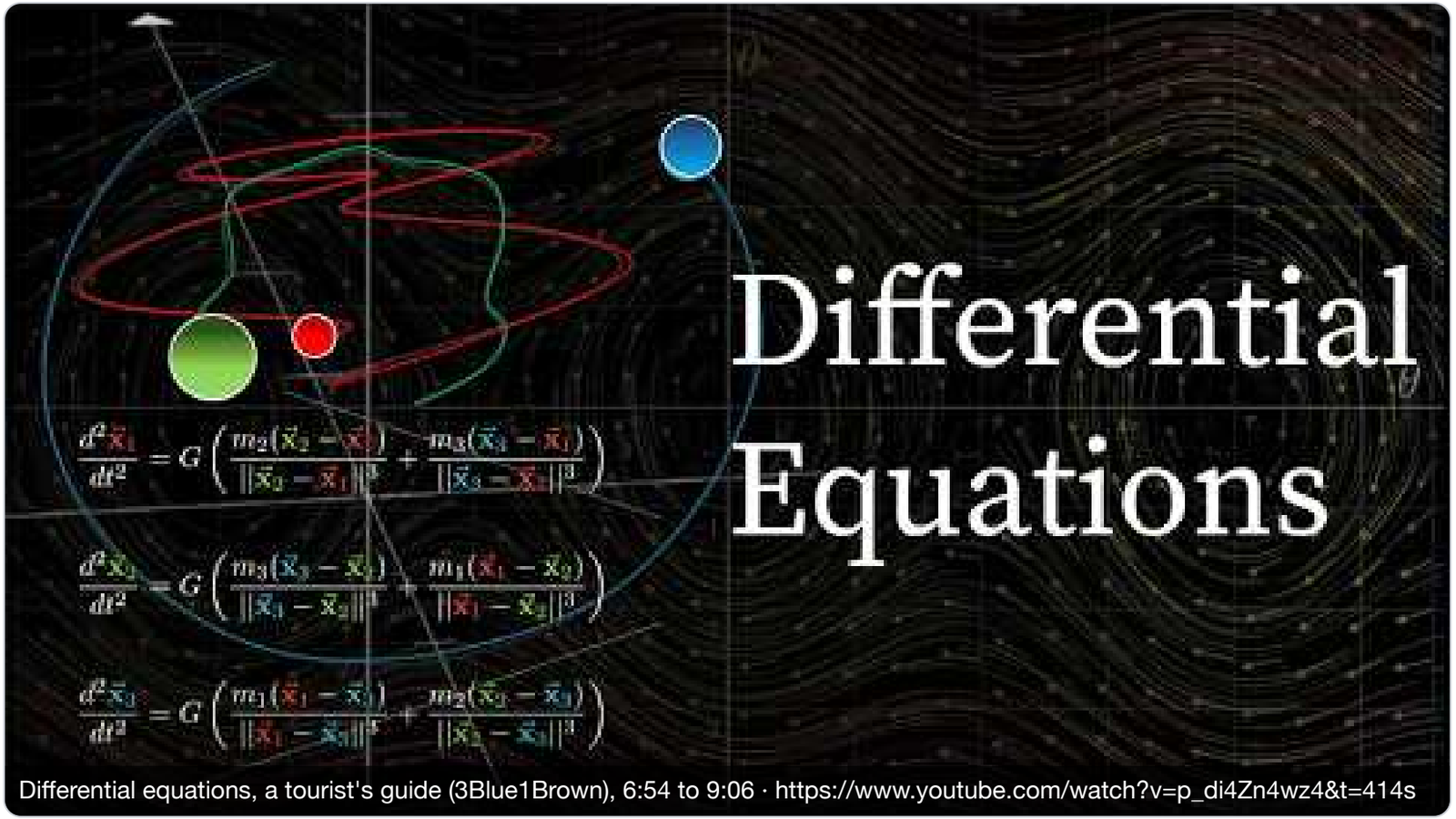
2 DoF: $\vec{q} = (\theta_1, \theta_2)$

Differential equations

A differential equation determines how the highest derivative depends on the lower ones:

$q^{(n)} = f\left(\vec{q}, \dot{\vec{q}}, \ldots, q^{(n-1)}, t\right)$, an ODE of order n

Example:



Differential equations, a tourist's guide (3Blue1Brown), 6:54 to 9:06 · https://www.youtube.com/watch?v=p_di4Zn4wz4&t=414s

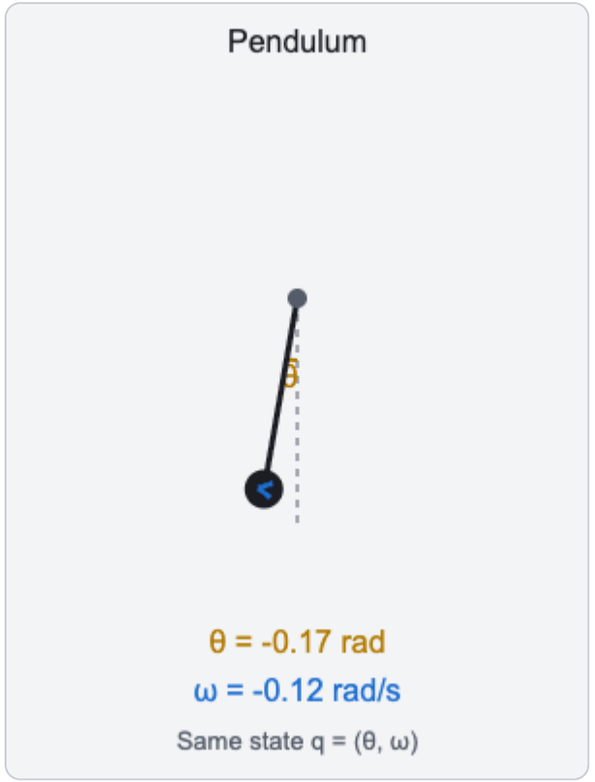
Every ODE can be made first order

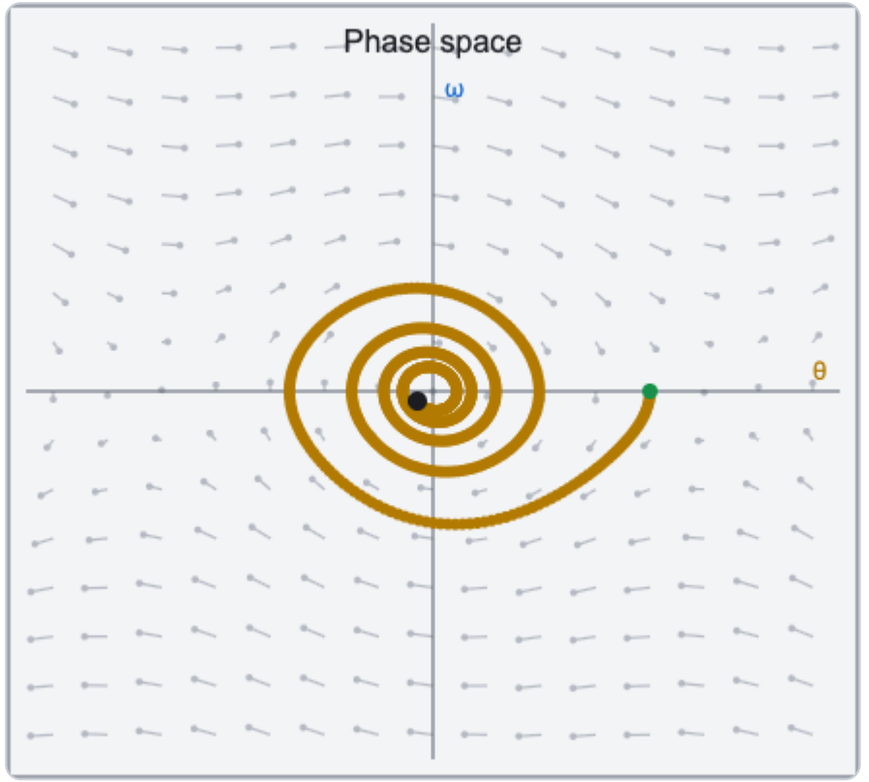
Any n-th order ODE can be made first order, by changing the state description:

$$\dot{\vec{q}} = f(\vec{q}(t), t)$$

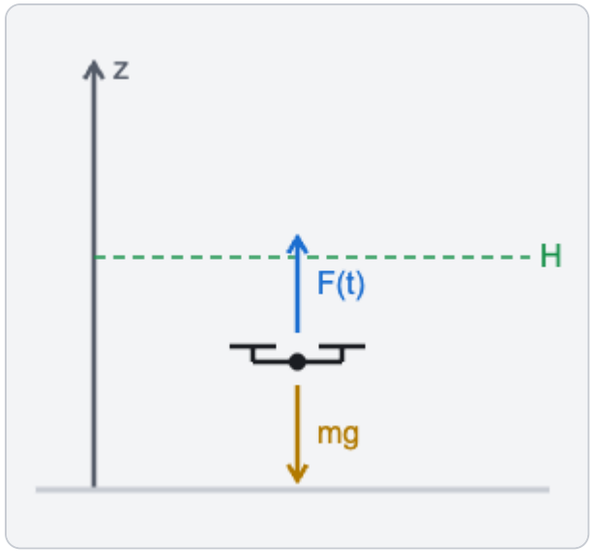
The pendulum: $\ddot{\theta} = -\frac{g}{L}\sin\theta - \mu\,\dot{\theta}$, state $\vec{q} = (\theta, \omega)$, $\omega = \dot{\theta}$:

$$\dot{\vec{q}} = \begin{bmatrix} \dot{\theta} \\ \dot{\omega} \end{bmatrix} = \begin{bmatrix} \omega \\ -\frac{g}{L}\sin\theta - \mu\,\omega \end{bmatrix} = f(\vec{q})$$





Differential equations for a drone



$$m\,\ddot{z} = F(t) - mg - \mu\,\dot{z}$$

- the thrust $F(t)$
- gravity
- air resistance

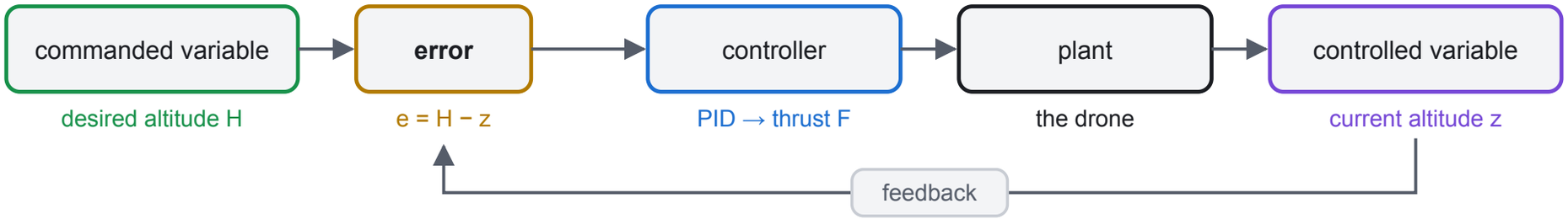
$$\mathbf{q} = (z, v), \quad v = \dot{z}, \qquad \dot{\mathbf{q}} = \begin{bmatrix} v \\ F(t)/m - g - (\mu/m)\,v \end{bmatrix}$$

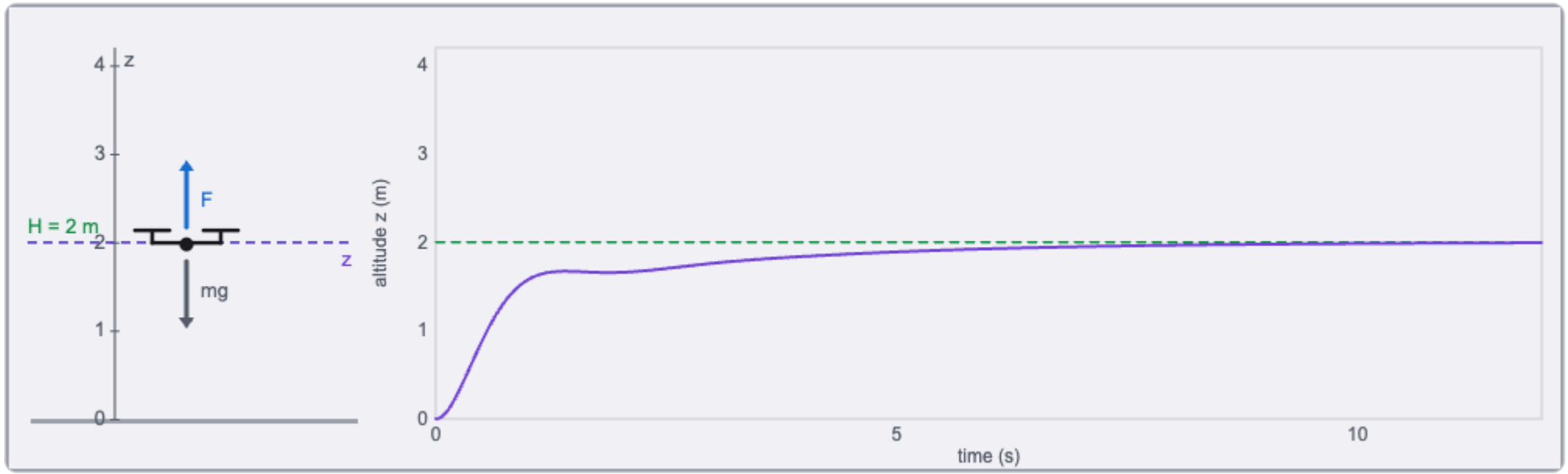
Question How do we choose the thrust $F(t)$ to reach the desired altitude H?

Part 2

PID control

A controller: the general idea





F = 9.8 N · mg = 9.8 N · z = 1.99 m · e = 0.01 m · t = 12 s

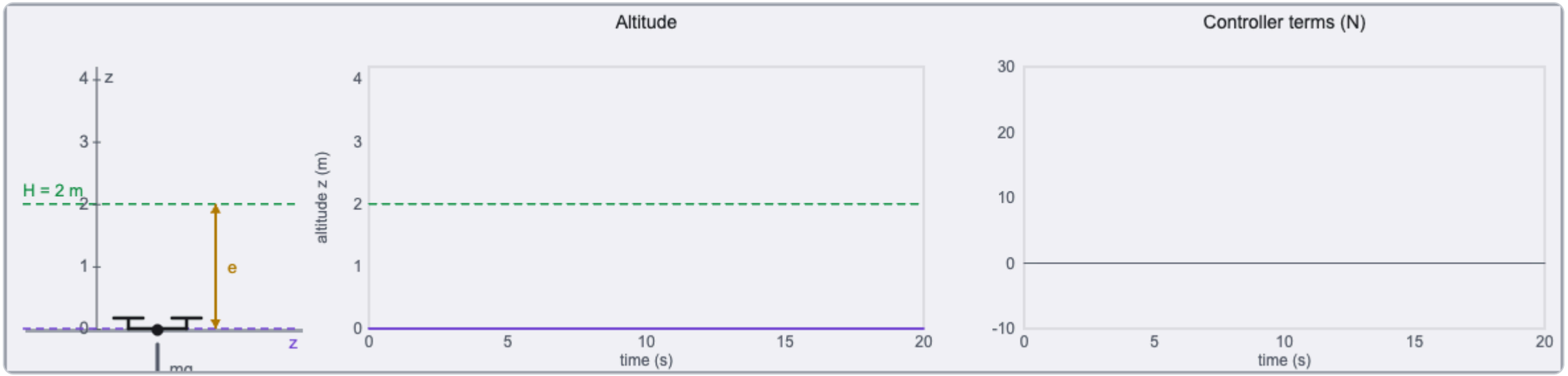
F = 12·e + 4·ė + 4·∫e · explicit Euler, Δt = 0.01

PID: proportional, integral, derivative

error
$e = H - z$

Proportional	what the error is now
Integral	the error accumulated over time
Derivative	how the error is changing

$$F_p = 0$$



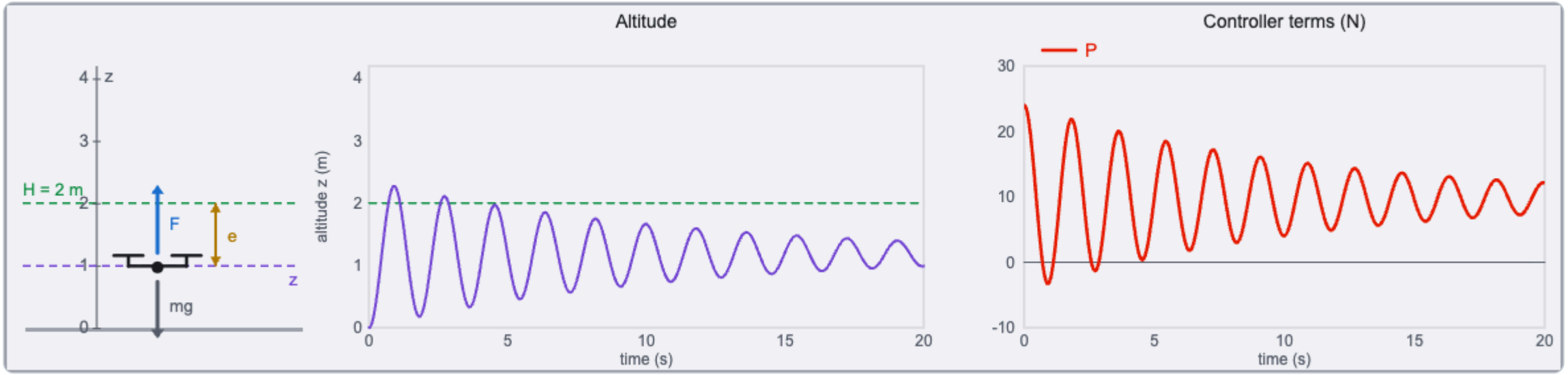
If some terms are missing, we have a P, PD, PI… controller.

The proportional term

error
$e = H - z$

Proportional	what the error is now	$k_p\, e$	**added**
Integral	the error accumulated over time		
Derivative	how the error is changing		

$$F_p = k_p\, e$$



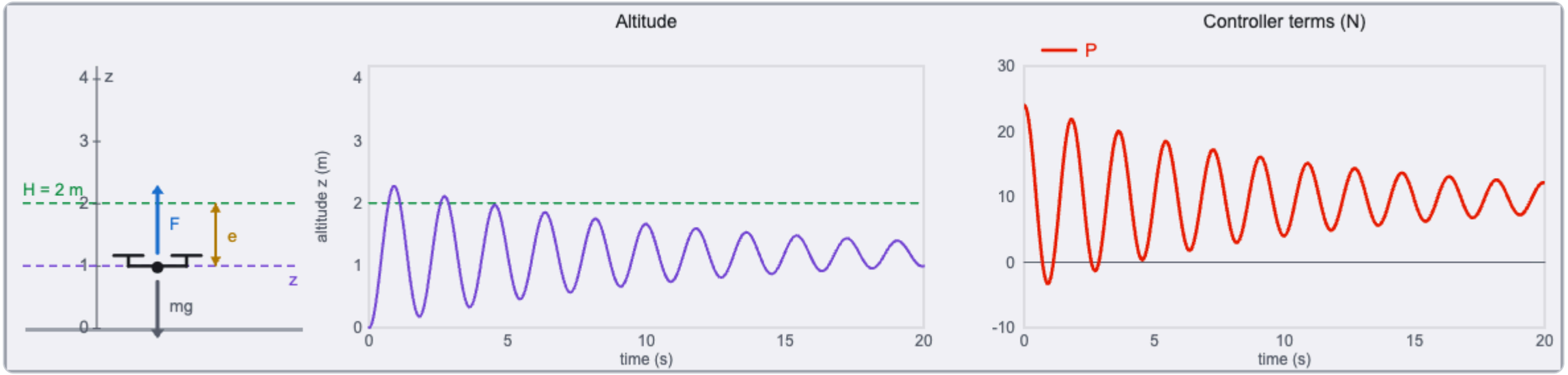
F = 12.1 N · mg = 9.8 N · z = 0.99 m · e = 1.01 m · t = 20 s

The P controller: shrinking oscillations

error $e = H - z$

Proportional	what the error is now	$k_p\, e$
Integral	the error accumulated over time	
Derivative	how the error is changing	

$$F_p = k_p\, e$$



F = 12.1 N · mg = 9.8 N · z = 0.99 m · e = 1.01 m · t = 20 s

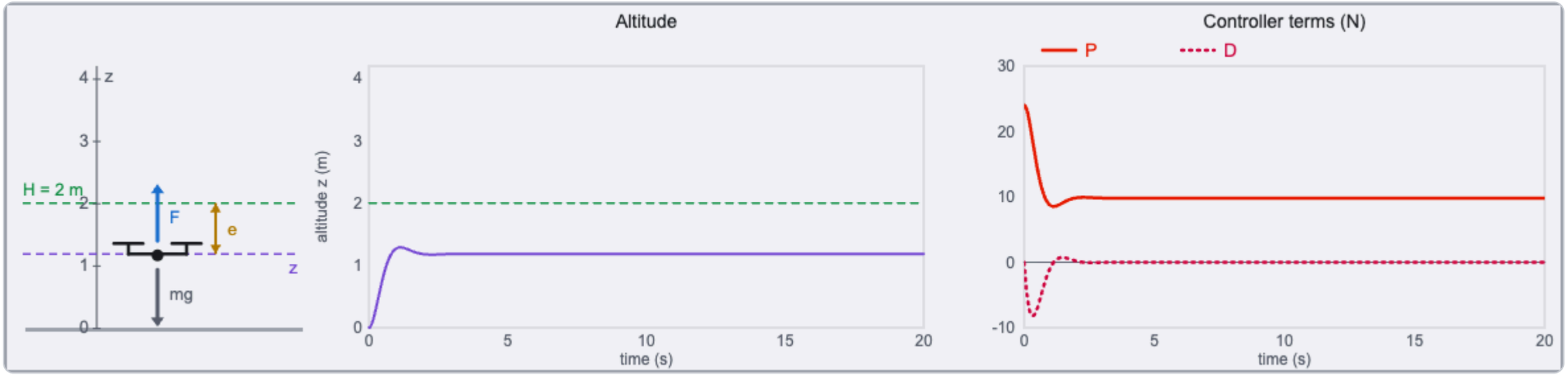
The amplitudes get **smaller and smaller** because of air resistance: $-\mu\, \dot{z}$. Let's use this to our advantage!

The derivative term

error $e = H - z$

Proportional	what the error is now	$k_p\, e$
Integral	the error accumulated over time	
Derivative	how the error is changing	$k_d\, \dot{e}$ **added**

$$F_p = k_p\, e + k_d\, \dot{e}$$



F = 9.8 N · mg = 9.8 N · z = 1.18 m · e = 0.82 m · t = 20 s

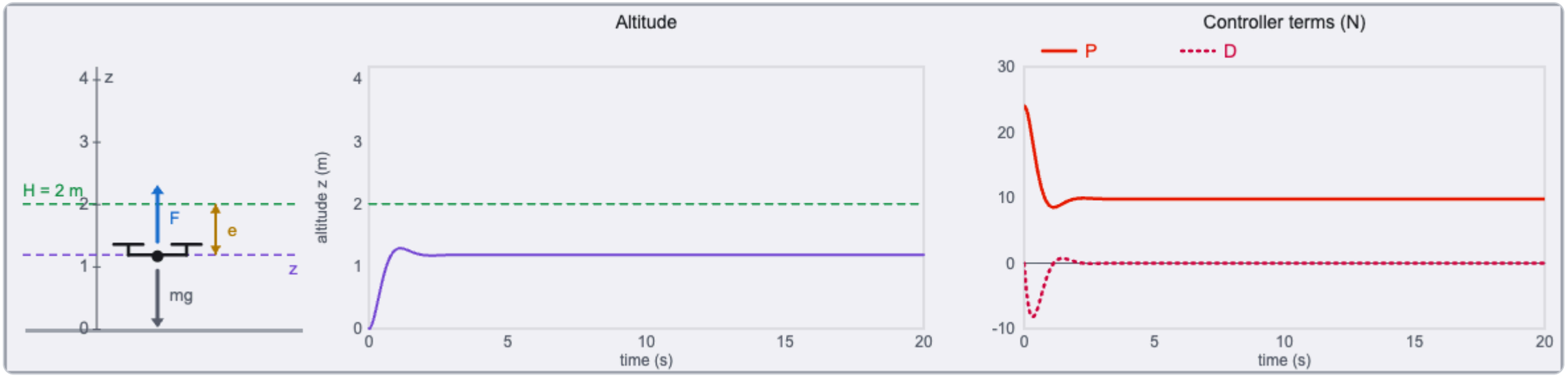
Our own “air resistance”: $\dot{e} = -\dot{z}$.

The PD controller: what happened

error $e = H - z$

Proportional	what the error is now	$k_p\, e$
Integral	the error accumulated over time	
Derivative	how the error is changing	$k_d\, \dot{e}$

$$F_p = k_p\, e + k_d\, \dot{e}$$



F = 9.8 N · mg = 9.8 N · z = 1.18 m · e = 0.82 m · t = 20 s

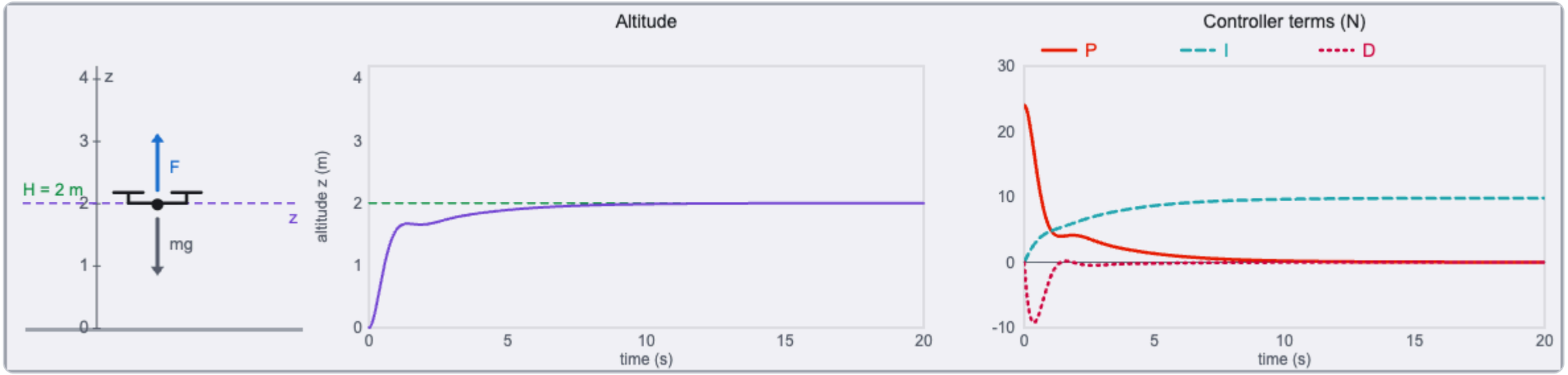
The drone settles **below** H, where $F_p = mg$. At H, with $e = 0$ and $\dot{e} = 0$, this controller would give $F_p = 0$.

The integral term

error $e = H - z$

Proportional	what the error is now	$k_p\, e$	
Integral	the error accumulated over time	$k_i \int e\, dt$	added
Derivative	how the error is changing	$k_d\, \dot{e}$	

$$F_p = k_p\, e + k_d\, \dot{e} + k_i \int e\, dt$$



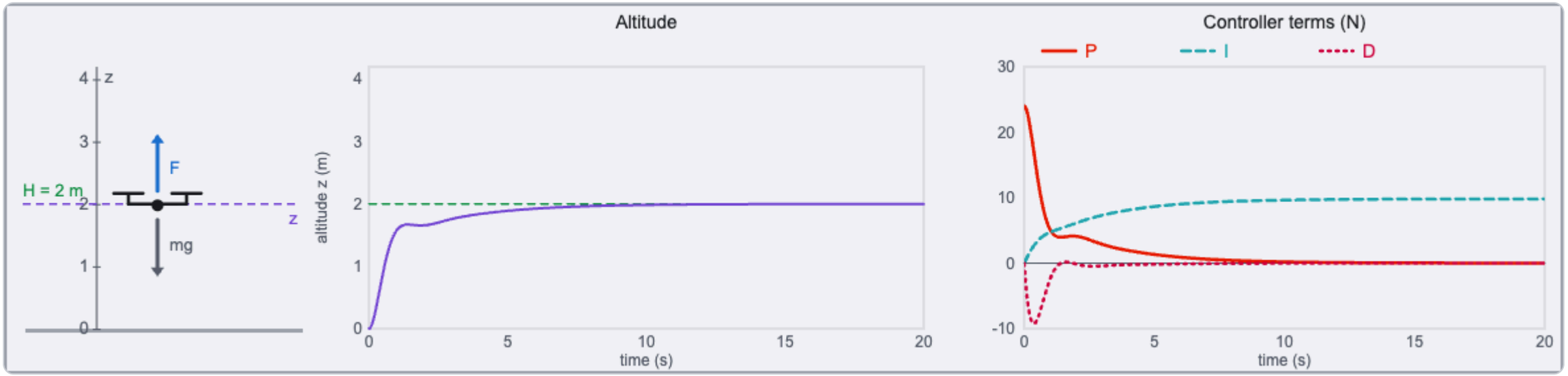
F = 9.8 N · mg = 9.8 N · z = 2 m · e = 0 m · t = 20 s

The PID controller: what happened

error $e = H - z$

Proportional	what the error is now	$k_p\, e$
Integral	the error accumulated over time	$k_i \int e\, dt$
Derivative	how the error is changing	$k_d\, \dot{e}$

$$F_p = k_p\, e + k_d\, \dot{e} + k_i \int e\, dt$$

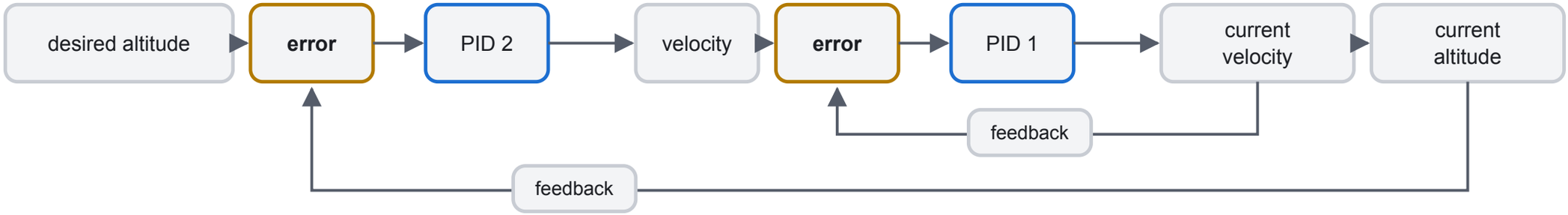


F = 9.8 N · mg = 9.8 N · z = 2 m · e = 0 m · t = 20 s

The drone reaches H. As e and $\dot{e}$ approach zero, the **accumulated integral term** supplies the thrust that balances gravity.

Several PID controllers at once

In real life we often have multiple PID controllers interacting with each other:



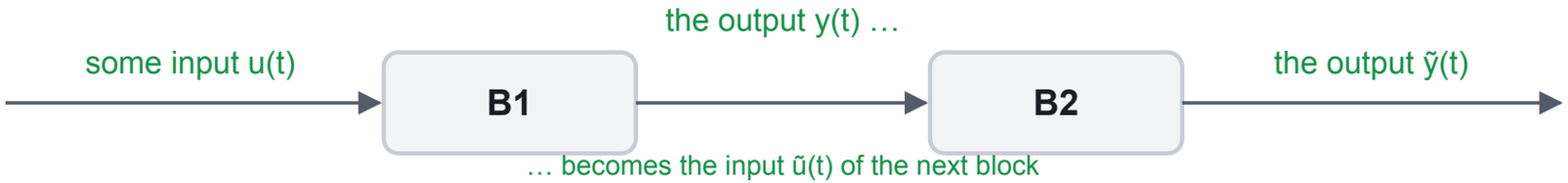
Why do this? For example, when one variable has to be controlled at a much higher frequency than the other.

- the rotor speed has to react quickly (e.g. to a gust of wind) to keep the desired velocity
- corrections to the altitude should come at a much slower rate, e.g. to average the measurement over time and get rid of the noise

Transfer functions

Transfer functions: the general idea

We started dealing with many interconnected blocks:

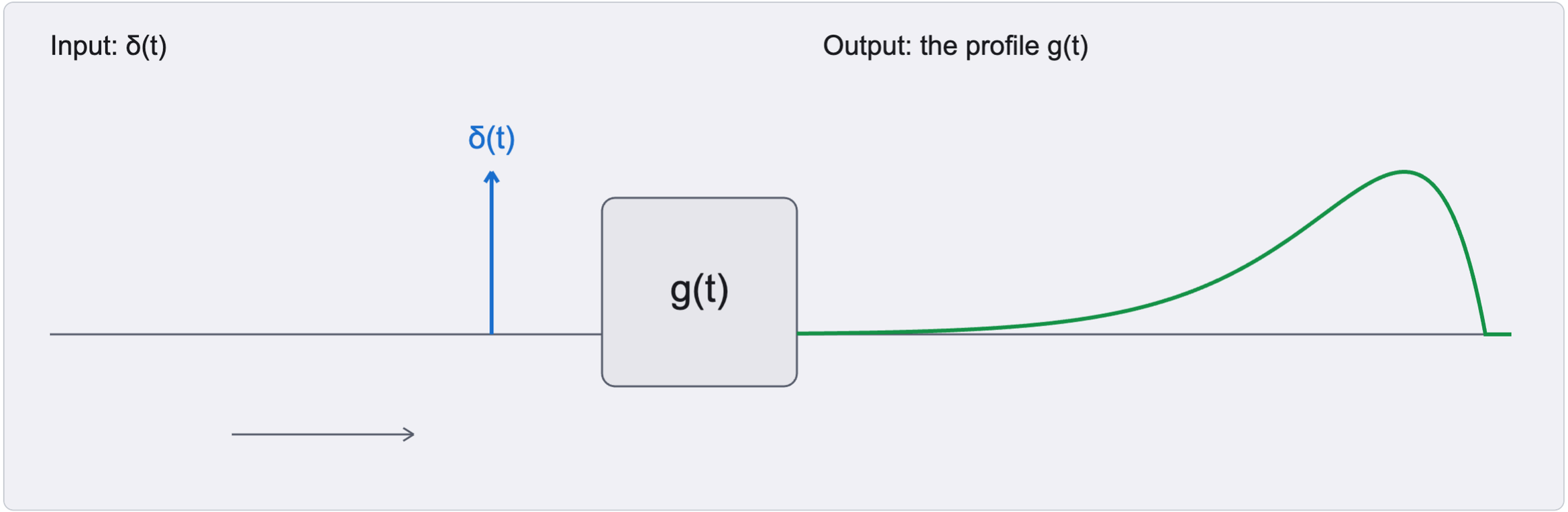


How to combine these consecutive signals?

Transfer functions: an efficient way to do it

- a vast topic: if you see it for the first time, do not try to process everything at once
- from the perspective of this course it is enough to treat it as a black-box tool

Without transfer functions: the profile of a block

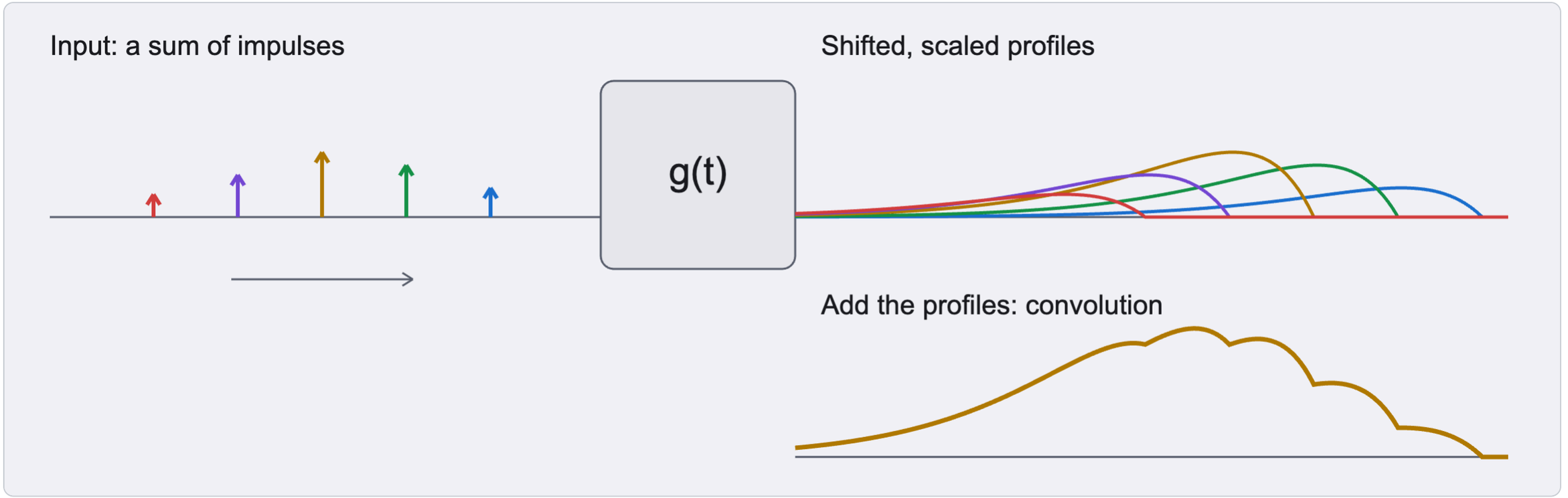


What is the profile of a block? Feed it a single impulse (a Dirac delta): the output it produces is the profile, $g(t)$. Also called the impulse response.

A Dirac delta: the limit of narrower and narrower bumps with the same area 1.

Without transfer functions: the output for any input

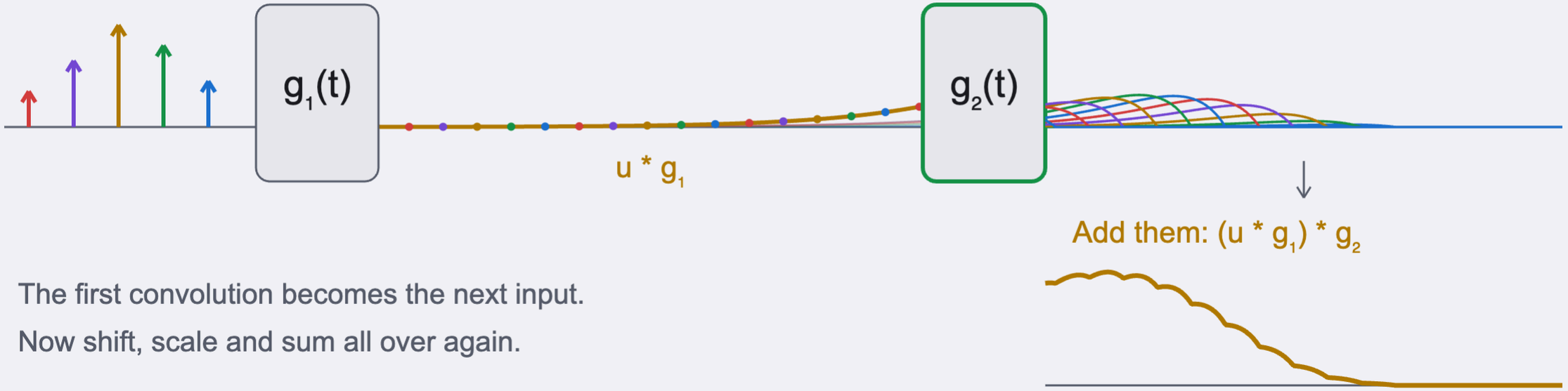
Split the input into impulses → shift and scale the profile → add the outputs.



$$y(t) = \int_0^t u(\tau)\, g(t-\tau)\, d\tau = (u * g)(t)$$

Convolution: the sum over all impulses.

The problem is…



$$y_2(t) = \int_0^t \underbrace{\left[\int_0^\tau u(\sigma)\, g_1(\tau - \sigma)\, d\sigma\right]}_{\text{output of the first block}} g_2(t - \tau)\, d\tau = (u * g_1 * g_2)(t)$$

Computing a convolution **of a convolution is hard**.

Transfer functions

It turns out we can use the **Laplace transform** $\mathcal{L}$:

$$\mathcal{L}u\,(s) = U(s) = \int_0^\infty u(t)\, e^{-st}\, dt$$

it represents all our functions by **unique** functions of a different variable, s

a weighted sum over time: what is the signal made of, exponentials and oscillations of which rates and frequencies?

$$\mathcal{L}\,(u * g) = \mathcal{L}u \cdot \mathcal{L}g$$

a **convolution** in the time domain is a **multiplication** of the representatives in the s domain

AND MULTIPLICATION IS EASY

Transfer functions: the PID controller

given an input represented in the s domain

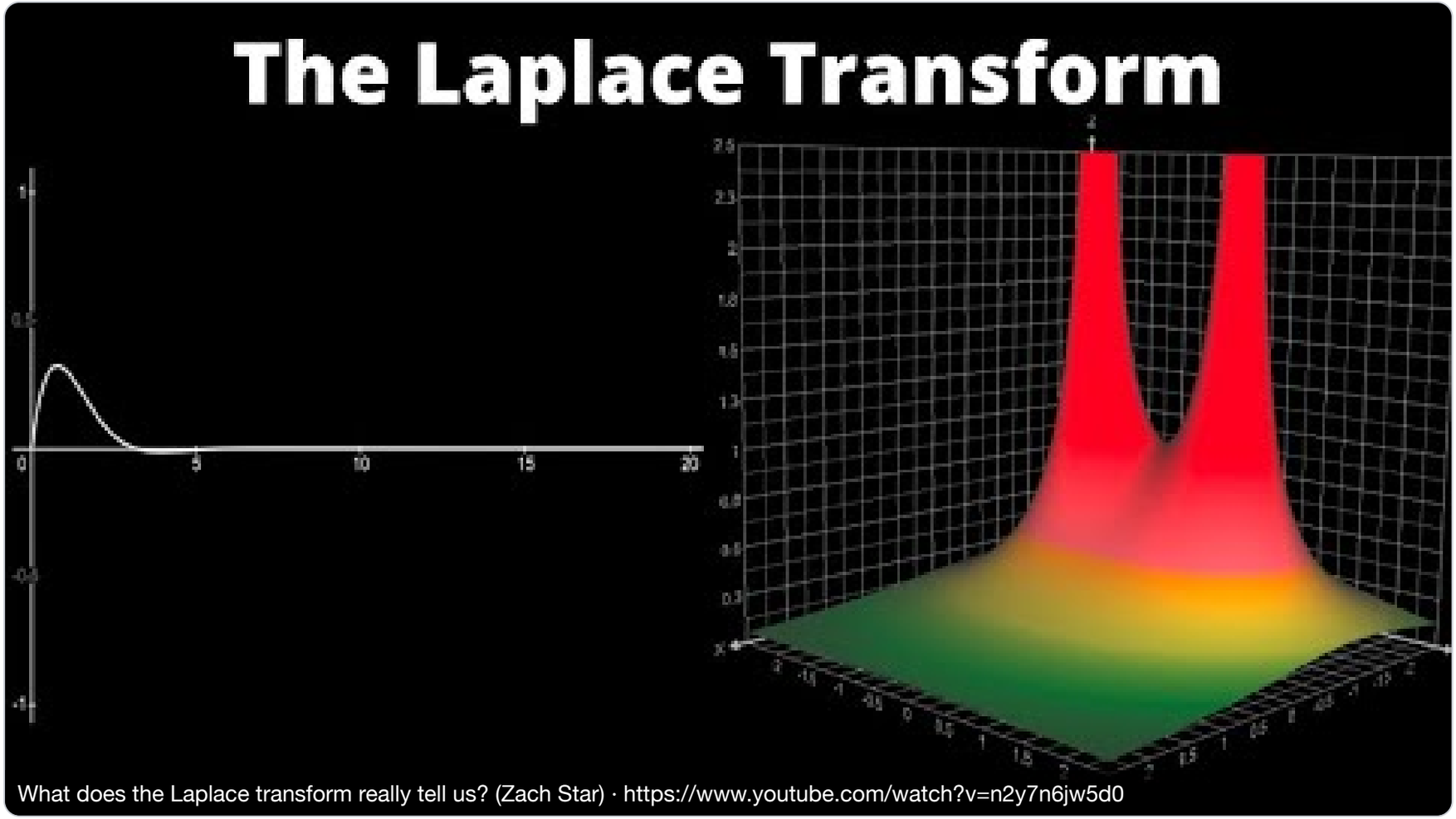
$$U(s)$$

	what we get in the s domain
Proportional	$k_p \cdot U(s)$
Integral	$\frac{k_i}{s} \cdot U(s)$
Derivative	$k_d\, s \cdot U(s)$

Integration becomes a division by s, differentiation a multiplication by s. The whole controller is one algebraic expression, $K(s) = k_p + k_i/s + k_d\, s$, and chaining blocks is multiplying such expressions.

Intuition behind the Laplace transform

OPTIONAL FURTHER READING

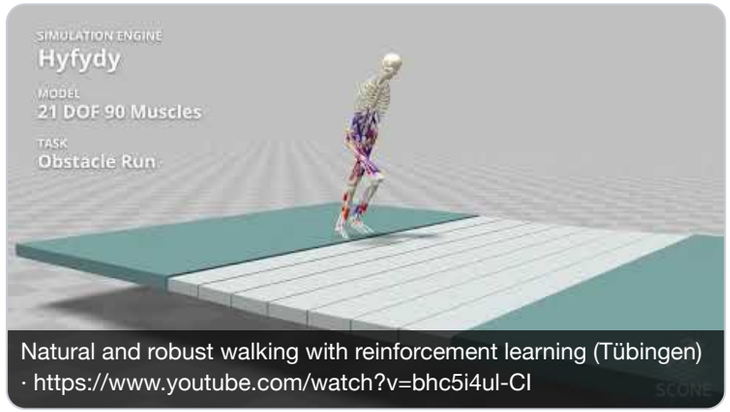


What does the Laplace transform really tell us? (Zach Star) · https://www.youtube.com/watch?v=n2y7n6jw5d0

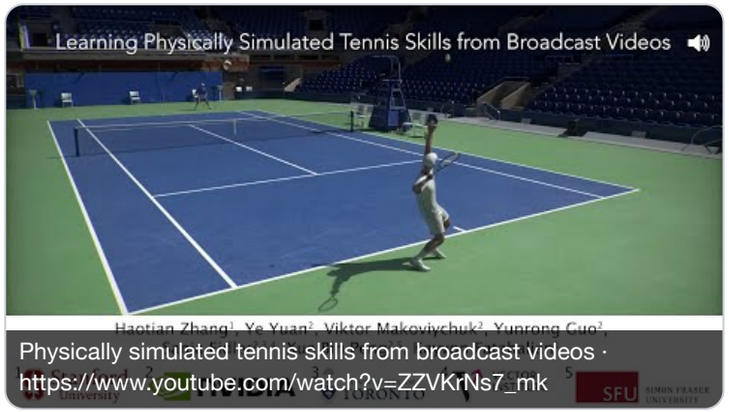
Part 3

Simulation engines

Examples and motivation



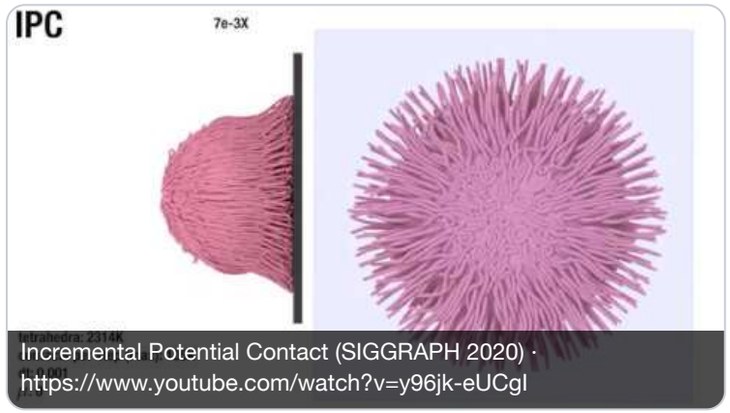
Natural and robust walking with reinforcement learning (Tübingen) · https://www.youtube.com/watch?v=bhc5i4ul-CI



Physically simulated tennis skills from broadcast videos · https://www.youtube.com/watch?v=ZZVKrNs7_mk



PhysGaussian: physics-integrated 3D Gaussians · https://www.youtube.com/watch?v=V96GfcMUH2Q [with audio]



Incremental Potential Contact (SIGGRAPH 2020) · https://www.youtube.com/watch?v=y96jk-eUCgI



Augmented IPC for sticky interactions · https://www.youtube.com/watch?v=k3e8ZMfLBRc



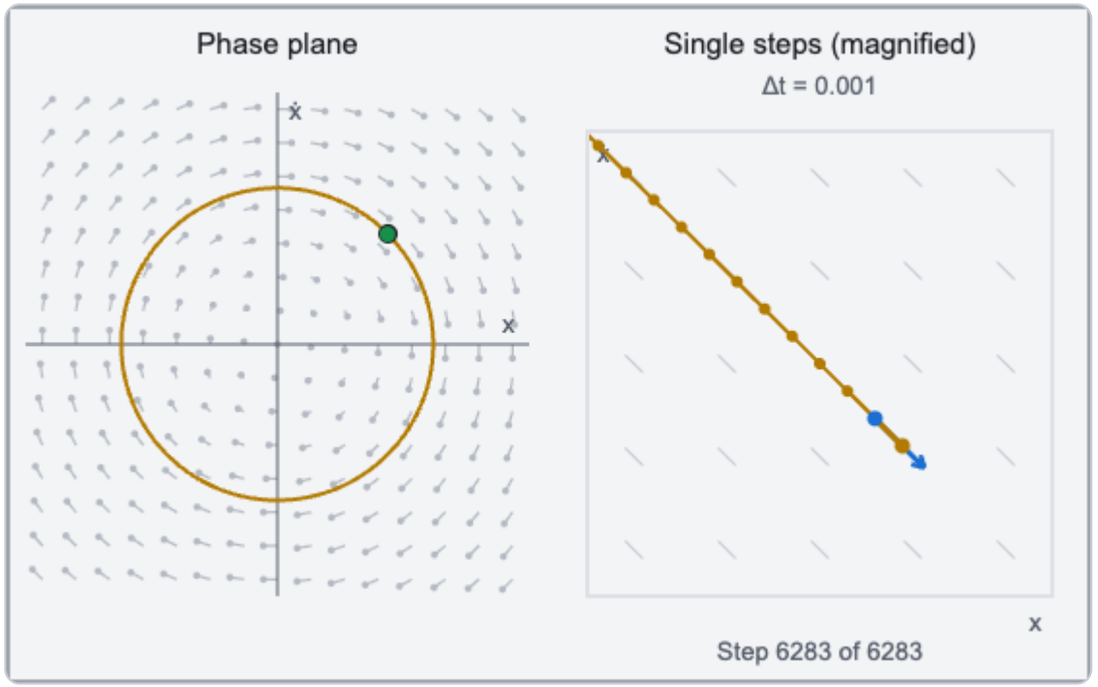
Physical Intelligence: π0 · https://www.youtube.com/watch?v=J-UTyb7lOEw

used a lot in **reinforcement learning**

can be combined with recent ML techniques such as **Gaussian splats**

the ideas are used in **diffusion and flow-matching** models

Simulation means numerical integration



1 · Start from the current state
$\mathbf{q} = (x, \dot{x})$

2 · Evaluate its rate of change
The arrow $f(\mathbf{q})$ at the current point

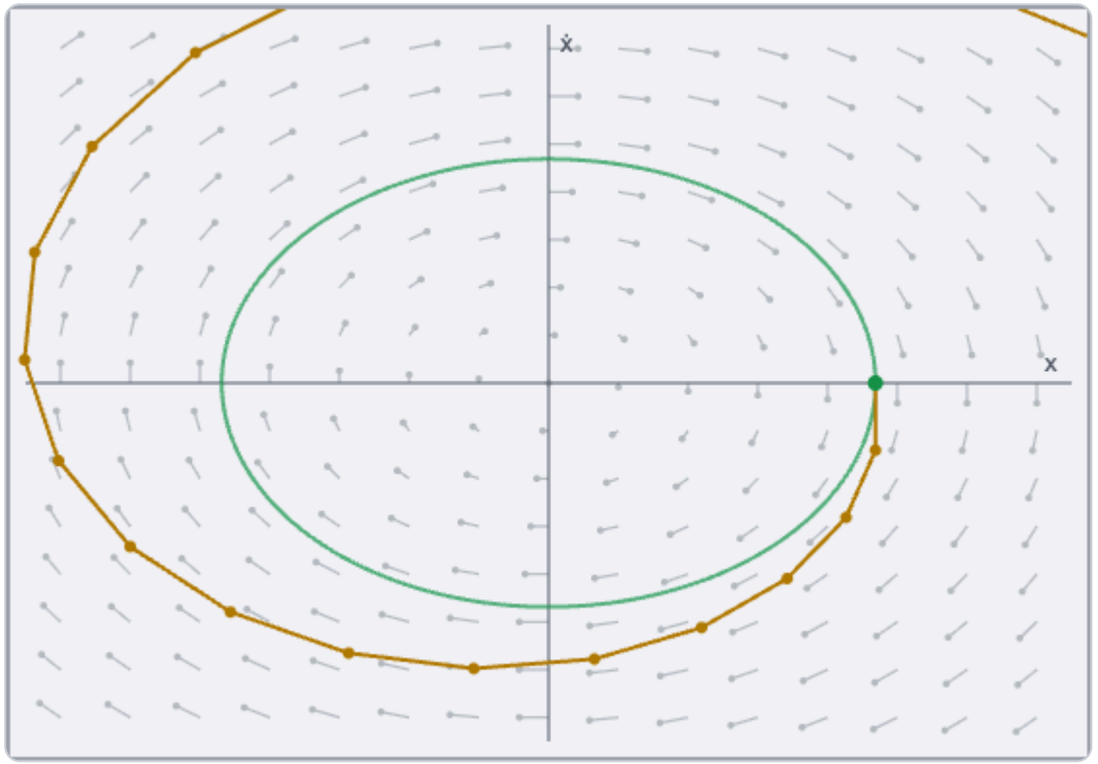
3 · Advance by a small Δt
Follow the arrow, then repeat from the new state.

Example: explicit Euler

$$\underbrace{f(\mathbf{q}, t) = \dot{\mathbf{q}}(t)}_{\text{DE of our system}} \approx \frac{\mathbf{q}(t + \Delta t) - \mathbf{q}(t)}{\Delta t} \quad \Longrightarrow \quad \mathbf{q}(t + \Delta t) \approx \mathbf{q}(t) + \Delta t\, f(\mathbf{q}(t), t)$$

Repeated updates trace how the state evolves.

What can go wrong in a simulation?



spring: ẍ = −x − c·ẋ, c = 0 (no damping) · click to choose the initial state

● explicit Euler, Δt = 0.3: step 80 of 80, t = 24 · — exact

A spring without damping: the exact trajectory is a closed orbit (green), the energy is conserved.

- every Euler step follows the arrow at the current point, which is a tangent of the orbit: it always lands slightly **outside**
- the errors add up: the state spirals outwards, the simulated spring gains energy from nothing
- make Δt smaller and compare

> The time step is not a detail: too large, and the simulation is wrong, or explodes.

More precise numerical schemes

There are different ways to approximate $\dot{\mathbf{q}}$. Start with the **Taylor expansion**:

$$\mathbf{q}(t \pm \Delta t) = \mathbf{q}(t) \pm \Delta t\, \dot{\mathbf{q}}(t) + \frac{\Delta t^2}{2}\ddot{\mathbf{q}}(t) \pm \frac{\Delta t^3}{6}\mathbf{q}^{(3)}(t) + \dots$$

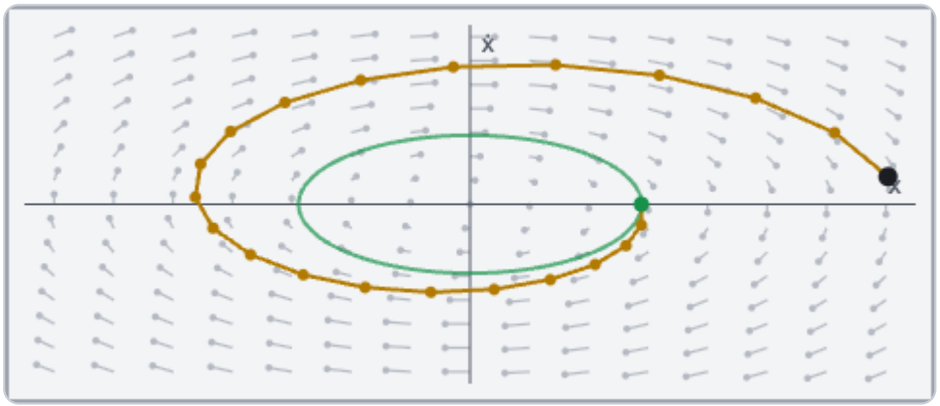
Forward Euler
$$\frac{\mathbf{q}(t+\Delta t) - \mathbf{q}(t)}{\Delta t} = \dot{\mathbf{q}}(t) + O(\Delta t)$$

Backward Euler
$$\frac{\mathbf{q}(t) - \mathbf{q}(t-\Delta t)}{\Delta t} = \dot{\mathbf{q}}(t) + O(\Delta t)$$

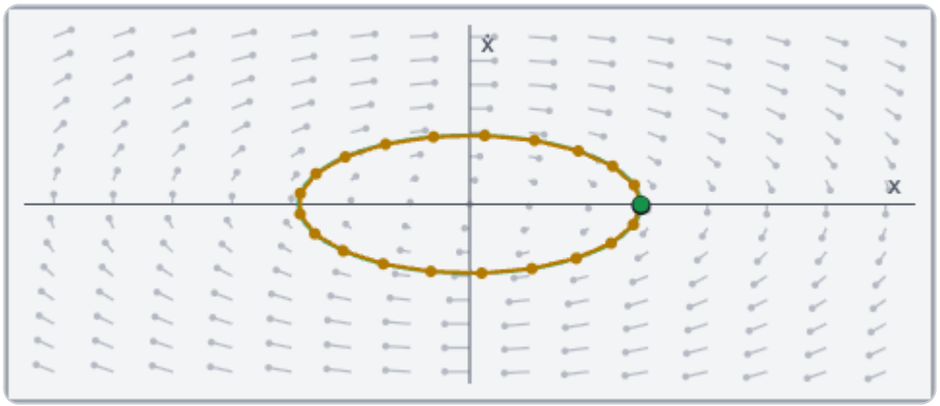
Central difference
$$\frac{\mathbf{q}(t+\Delta t) - \mathbf{q}(t-\Delta t)}{2\Delta t} = \dot{\mathbf{q}}(t) + O(\Delta t^2)$$

Subtracting the two Taylor expansions cancels the even powers. The central difference's error scales as Δt^2: **halve the step → quarter the error**.

Explicit Euler · $\Delta t = 0.3$



Runge–Kutta (RK4) · $\Delta t = 0.3$

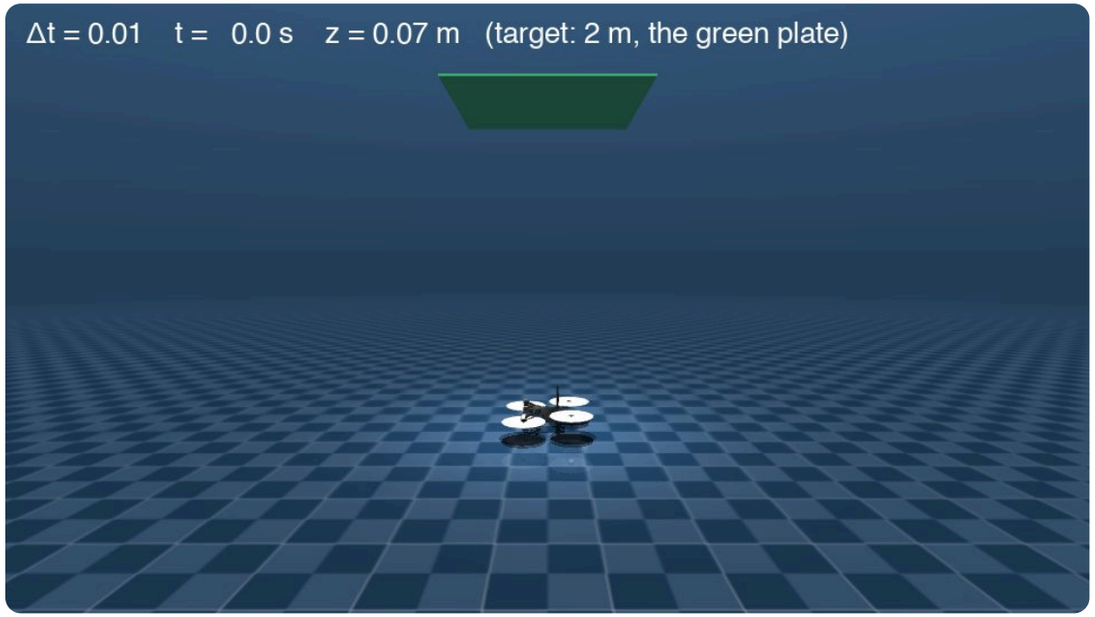


RK4 uses four rate evaluations per step: smaller error at the same Δt. Green: exact trajectory.

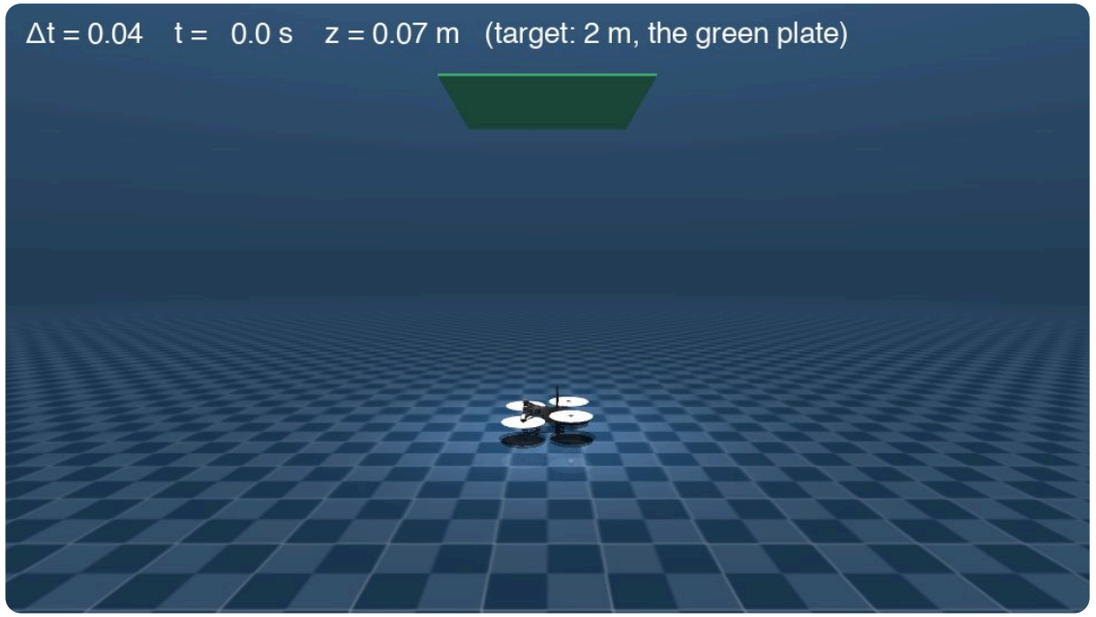
A wrong time step for the P-controlled drone

The drone simulator used in the lab sessions, with a P controller on the altitude and nothing else changed between the two runs:

the correct version: $\Delta t = 0.01$



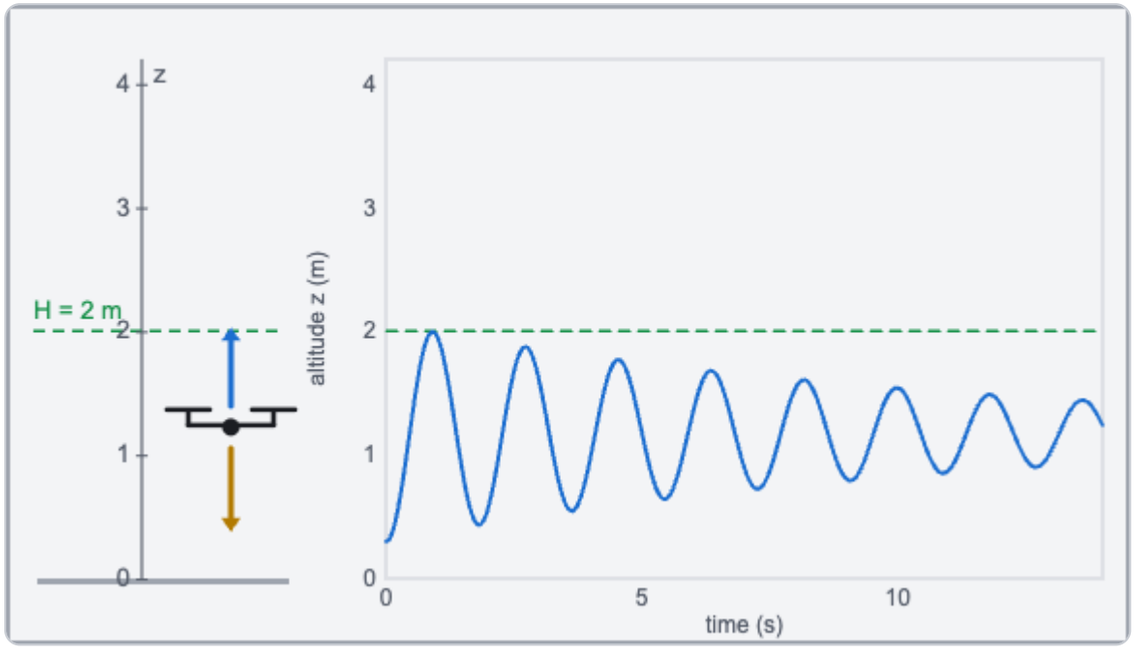
the time step increased just to $\Delta t = 0.04$



With the larger step the oscillation grows instead of dying out, until the drone hits the floor and breaks. The drone, the controller and the gains did not change.

A wrong time step for the P-controlled drone: the plot

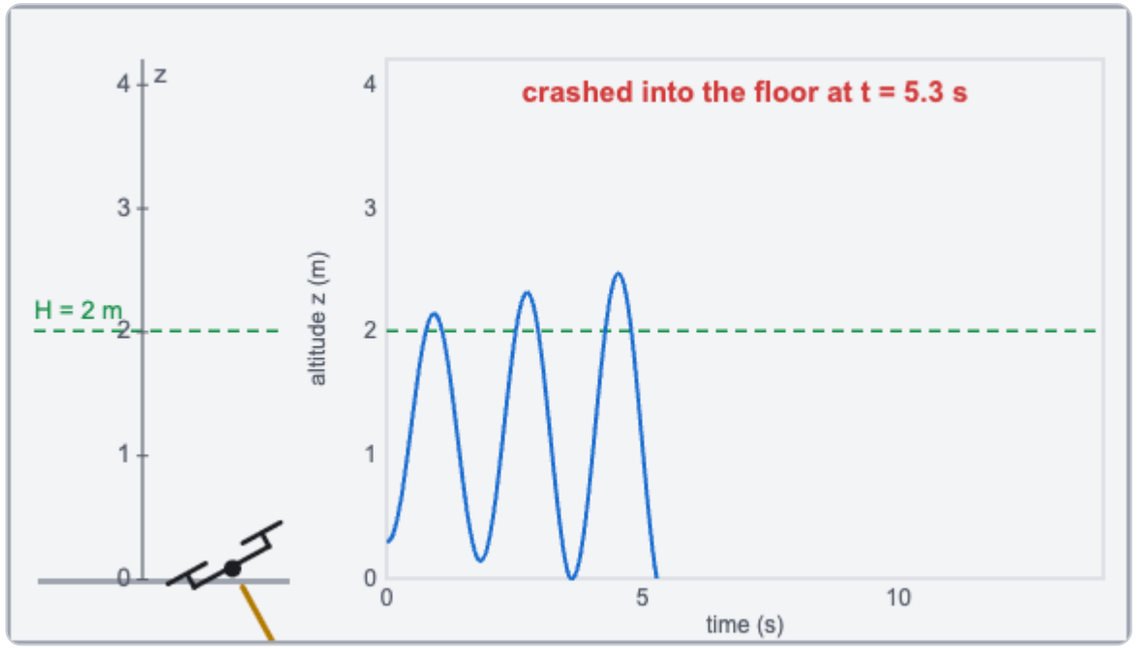
the correct version: $\Delta t = 0.01$



F = 9.1 N · mg = 9.8 N · z = 1.24 m · e = 0.76 m · t = 14 s

F = 12·e · explicit Euler, Δt = 0.01

the time step increased just to $\Delta t = 0.04$



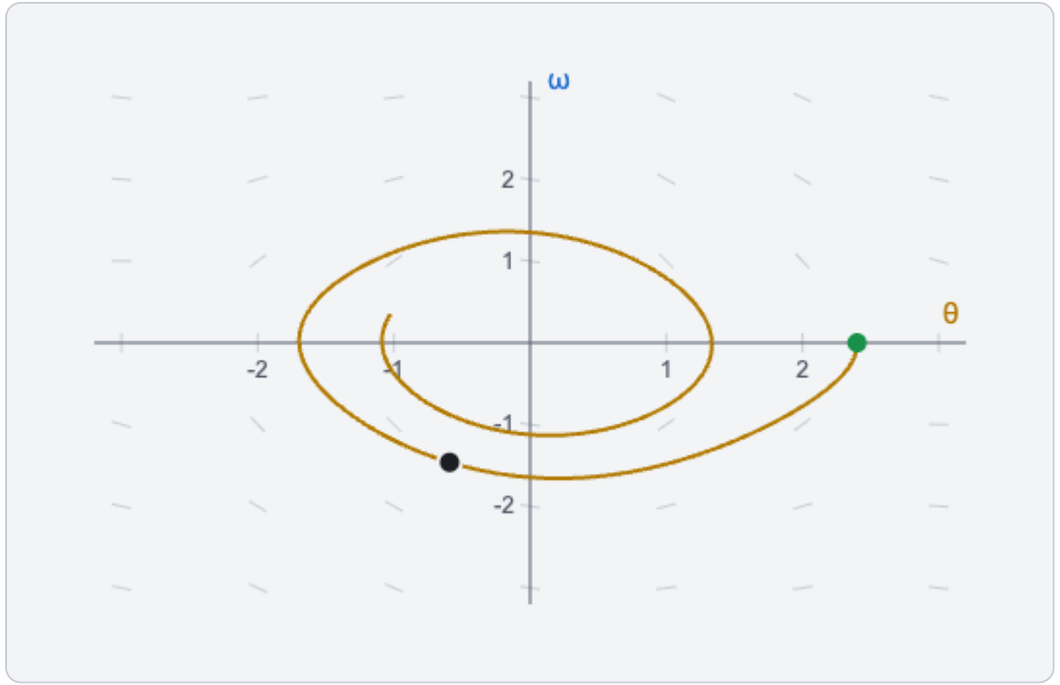
F = 24 N · mg = 9.8 N · z = 0 m · e = 2 m · t = 5.3 s

F = 12·e · explicit Euler, Δt = 0.04

Our toy simulation, starting at $z = 0.3$ m and played at $0.75\times$ speed: the oscillation grows until the drone lands too hard. Try RK4 with the same Δt: a better integrator survives the larger step.

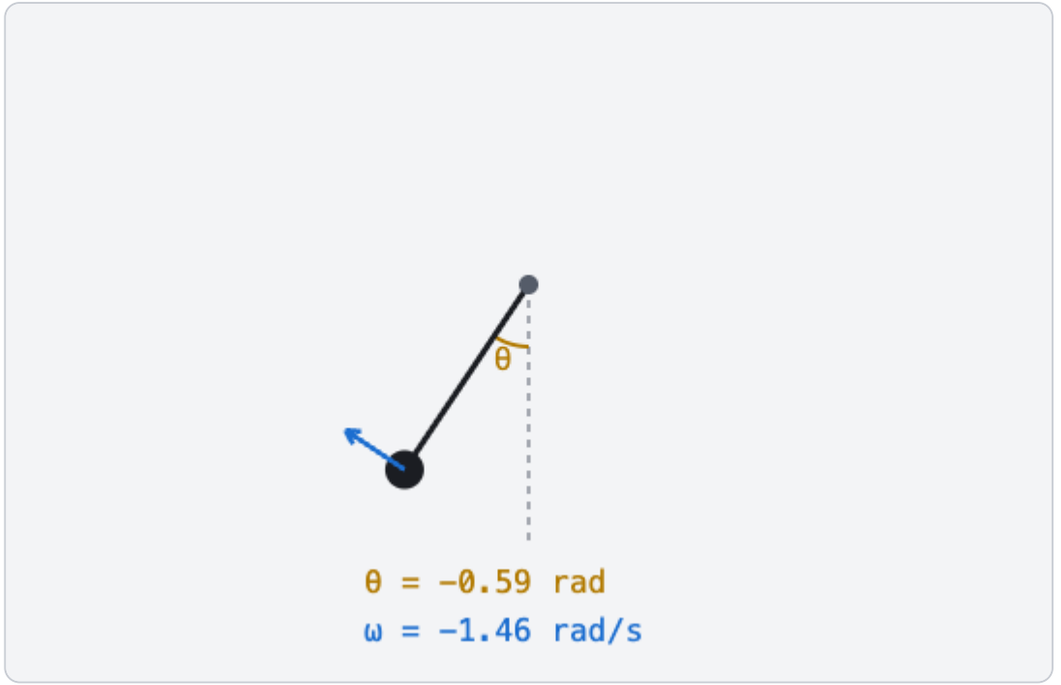
A note on the rendering speed

1 · Precompute the trajectory



241 stored states · 12 s · explicit Euler

2 · Play back the stored states

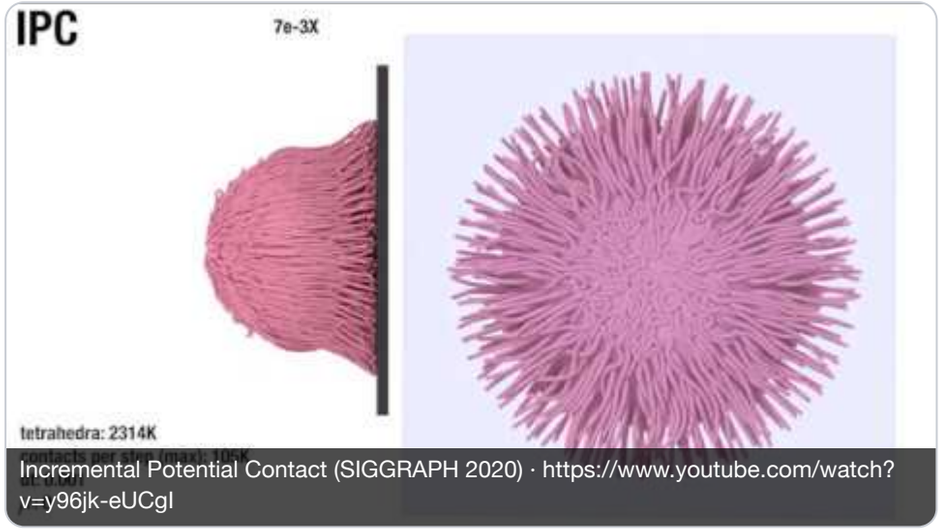


t = 3.00 s · state 61 / 241
Wall time: 0.00 s · full run: 12.00 s

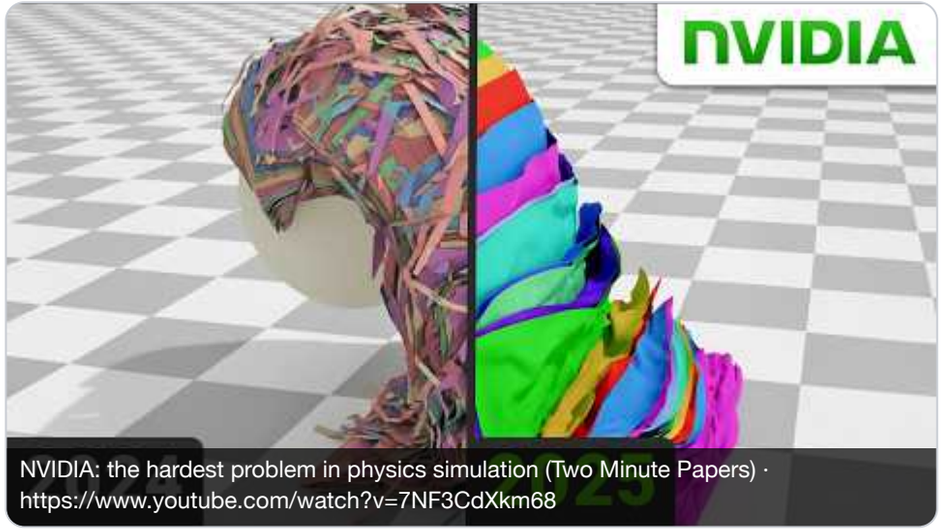
At 1× we display 20 stored steps/s. Increasing Δt from 0.05 to 0.2 s or using 4× playback both shorten the full run from **12 to 3 wall-clock seconds**. Only changing Δt changes the trajectory.

Extensions, Applications & Discussion

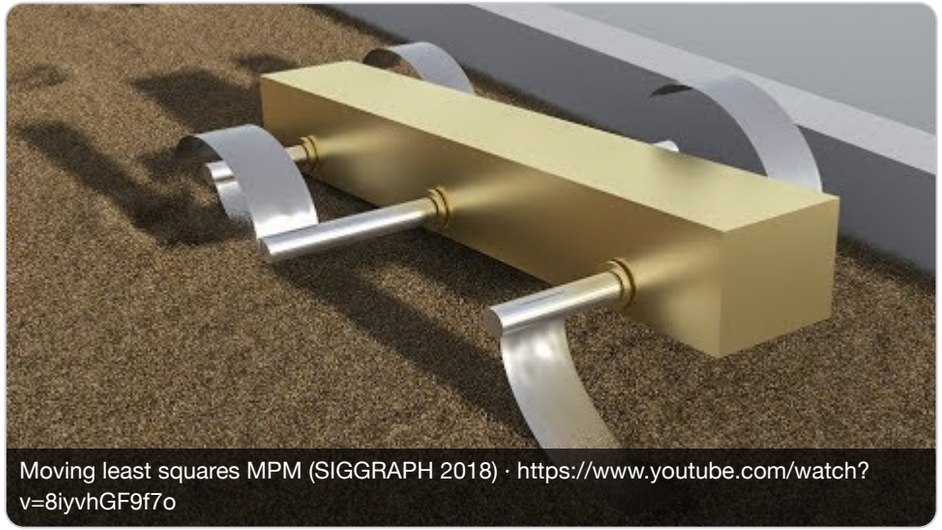
Simulating deformation

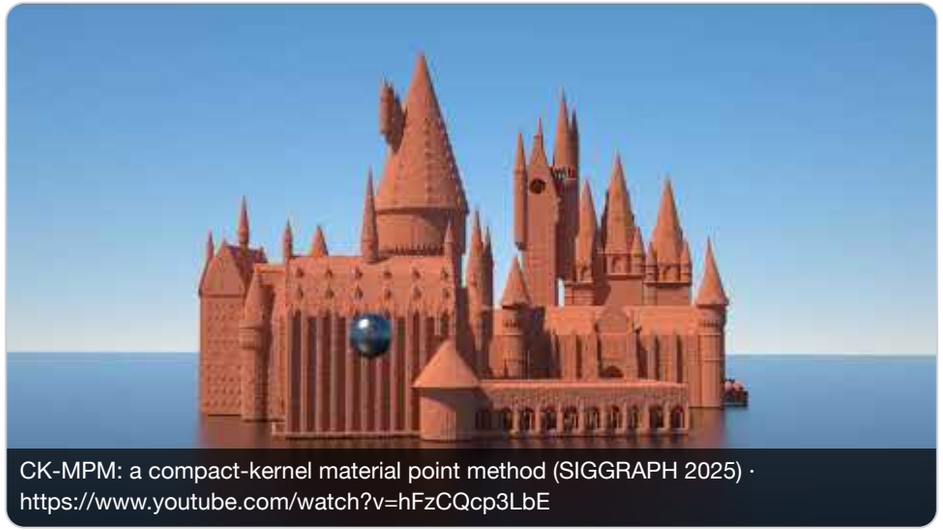


Incremental Potential Contact (SIGGRAPH 2020) · https://www.youtube.com/watch?v=y96jk-eUCgI



NVIDIA: the hardest problem in physics simulation (Two Minute Papers) · https://www.youtube.com/watch?v=7NF3CdXkm68

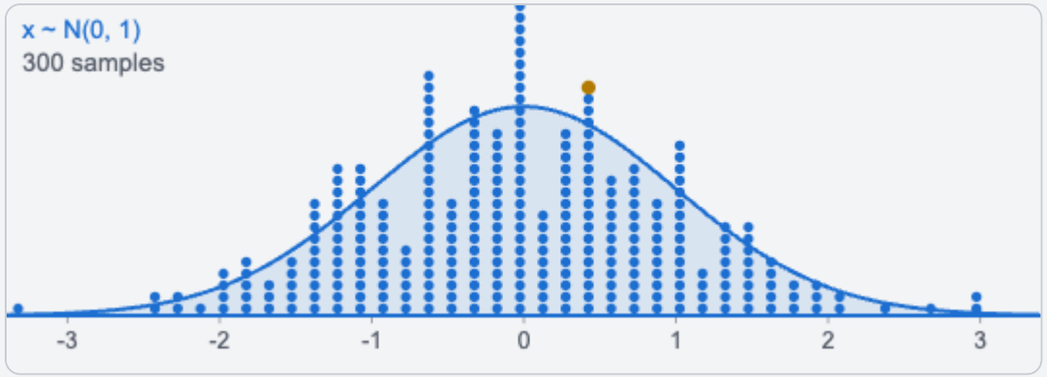

Moving least squares MPM (SIGGRAPH 2018) · https://www.youtube.com/watch?v=8iyvhGF9f7o


CK-MPM: a compact-kernel material point method (SIGGRAPH 2025) · https://www.youtube.com/watch?v=hFzCQcp3LbE

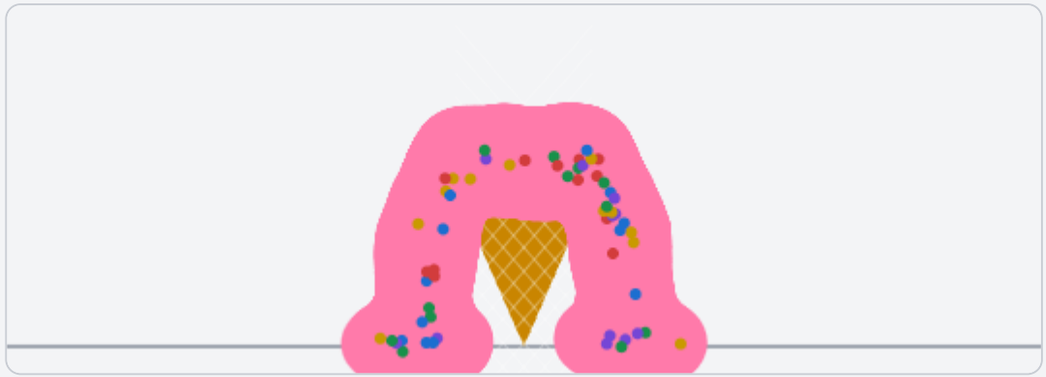
Cloth, jelly, snow, sand, fluids: a continuous body is cut into small parts, and the simulator keeps track of how each part moves and deforms. A field of its own; simulation engines do it for you.

Flow Matching & Diffusion: Deforming probability

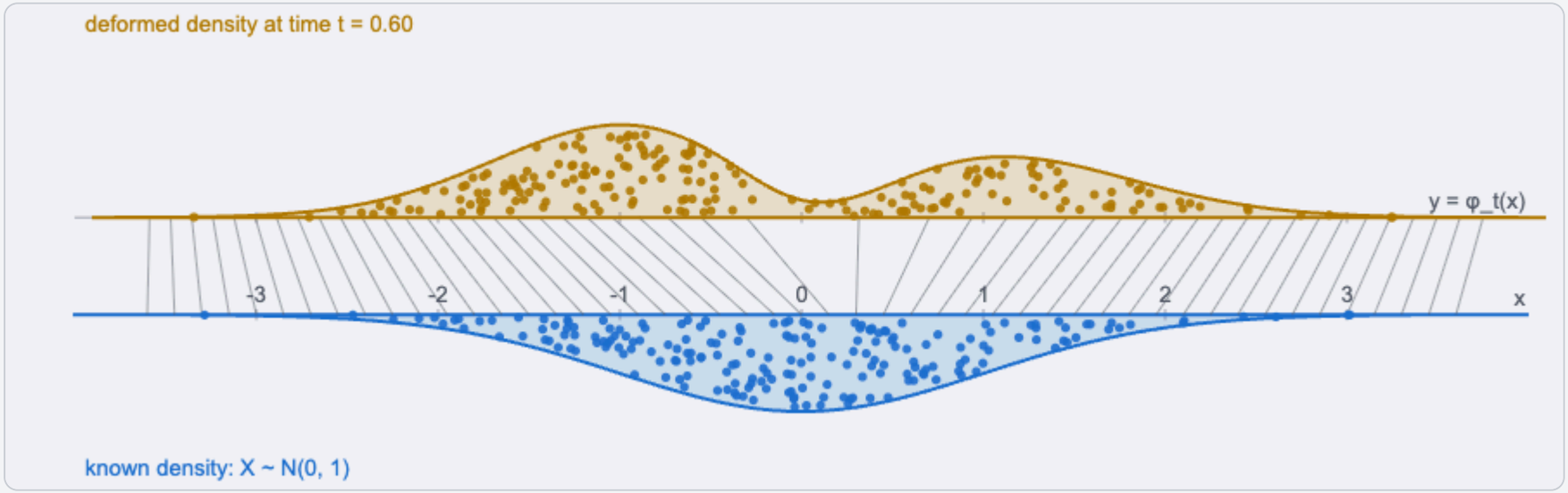
We can easily sample from known distributions



Deformable simulations: studied for decades



Combine the two ideas to generate samples from the distribution of interest



Example application: π0 - idea

Flow Matching

$$\mathbf{A}_t^0 \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

$$\downarrow$$

$$\cdots$$

$$\downarrow$$

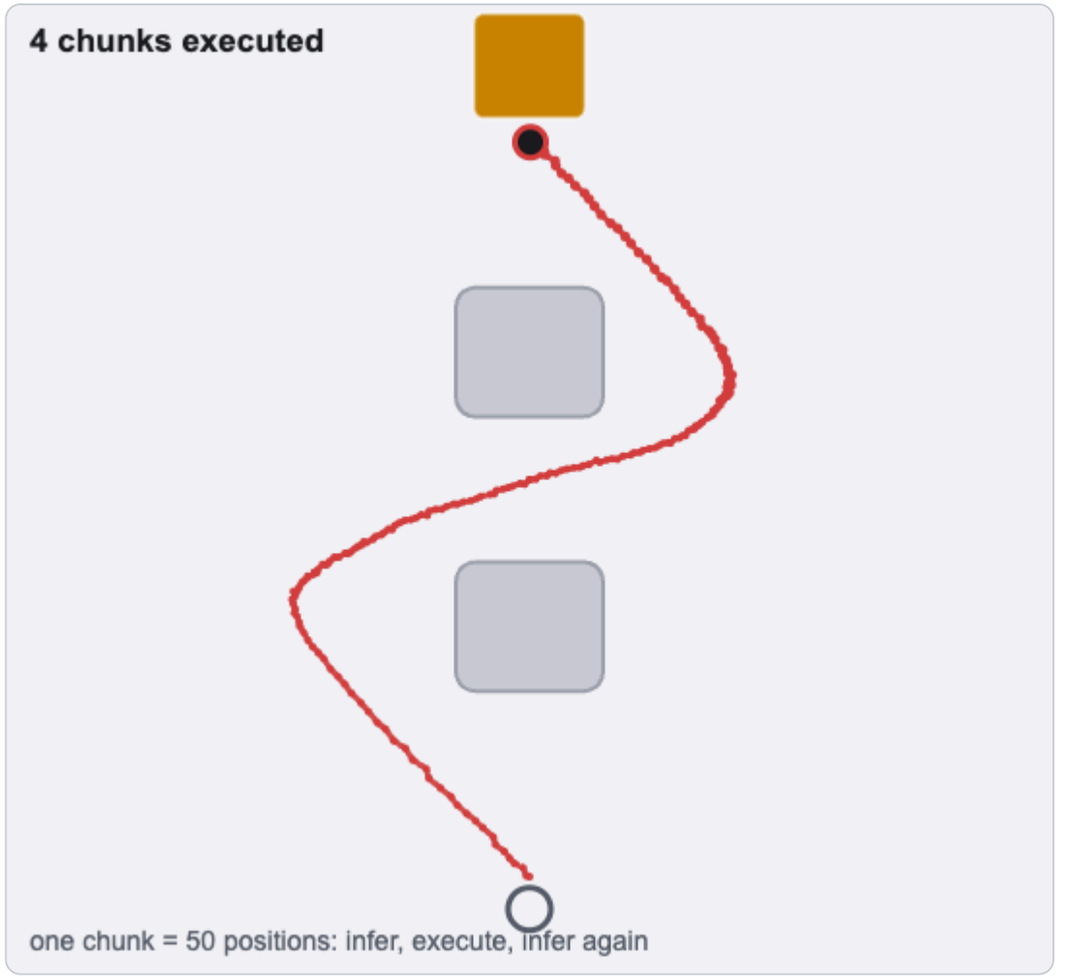
$$\mathbf{A}_t^\tau$$

$$\downarrow$$

$$\cdots$$

$$\downarrow$$

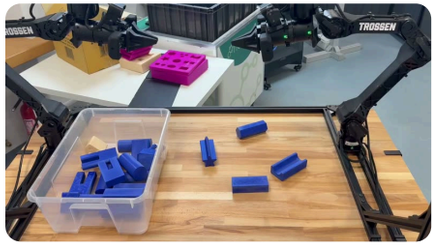
$$\mathbf{A}_t^1 \sim p_1$$



In action

Also at MIMUW :)







Physical Intelligence · π0.7



At MIMUW: flow-matching and diffusion policies for bimanual manipulation, trained from demonstrations. If you want to work on this, talk to us.

Summary

- a system is a **state** and a **vector field** $\dot{\mathbf{q}} = f(\mathbf{q}, t)$
- constraints reduce the degrees of freedom
- every ODE can be made first order
-
- **feedback control**: measure, compare with the target, act on the error, repeat
- **PID**: act on the error now (P), on its accumulated past (I), on its trend (D)
- controllers are often **cascaded**: a fast inner loop, a slow outer one
-
- a simulator **integrates** the field step by step, and the time step + numerical scheme decide the accuracy
- rendering speed does not change the physics, the numerical integration timestep does

References and more on these topics

PID control

- Brian Douglas, lectures on PID control (playlist)
- Brian Douglas, transfer functions
- Zach Star, the Laplace transform, a visual explanation
- S. W. Smith, convolution, chapter 6 of The Scientist and Engineer's Guide to DSP

Differential equations, simulation engines & flow matching

- 3Blue1Brown, Differential equations, a tourist's guide
- Steve Brunton, numerical differentiation and integration (playlist)
- C. Jiang et al., The material point method for simulating continuum materials (SIGGRAPH course)
- MuJoCo
- For flow matching: see my DNN lecture and the references there

Feedback



Please spend a minute: your feedback helps us improve the next lecture.

`tinyurl.com/mimuw-rc-feedback`

Anonymous, no login, three questions.